Computers and Chemical Engineering 204 (2026) 109418

Contents lists available at ScienceDirect

Computers and Chemical Engineering

journal homepage: www.elsevier.com/locate/cace

Physics-informed neural networks with hard nonlinear equality and inequality constraints

Ashfaq Iftakher [a], Rahul Golder [a], Bimol Nath Roy [a], M.M. Faruque Hasan [a,b,*]

[a] Artie McFerrin Department of Chemical Engineering, Texas A&M University, College Station, TX 77843-3122, USA
[b] Texas A&M Energy Institute, Texas A&M University, College Station, TX, 77843, USA

ARTICLE INFO

Keywords:
Machine learning
Constrained learning
Physics-informed neural network
Surrogate modeling

ABSTRACT

Traditional physics-informed neural networks (PINNs) do not guarantee strict constraint satisfaction. This is problematic in engineering systems where minor violations of governing laws can degrade the reliability and consistency of model predictions. In this work, we introduce KKT-Hardnet,[1] a neural network architecture that enforces linear and nonlinear equality and inequality constraints up to machine precision. It leverages a differentiable projection onto the feasible region by solving Karush-Kuhn-Tucker (KKT) conditions of a distance minimization problem. Furthermore, we reformulate the nonlinear KKT conditions via a log-exponential transformation to construct a sparse system with linear and exponential terms. We apply KKT-Hardnet to a nonconvex pooling problem and a real-world chemical process simulation. Compared to multilayer perceptrons and PINNs, KKT-Hardnet achieves strict constraint satisfaction. It also circumvents the need to balance data and physics residuals in PINN training. This enables the integration of domain knowledge into machine learning towards reliable hybrid modeling of complex systems.

1. Introduction

Neural networks have gained widespread popularity as surrogate models that enable fast approximate predictions of complex physical systems. Among them, physics-informed neural networks (PINNs) have emerged as a powerful technique to embed first principles-based physical knowledge directly into the learning process. Unlike purely data-driven models, PINNs integrate known physical constraints into the training objective, typically by augmenting the loss function with penalty terms that account for constraint violations (Karniadakis et al., 2021). These constraints may include algebraic equations (Iftakher et al., 2025a), ordinary and partial differential equations (Eivazi et al., 2022; Chen et al., 2021; Raissi et al., 2020), or combinations thereof. Although such high-fidelity models mimic the underlying physics, they are often computationally prohibitive, particularly in practical scenarios, such as real-time control (Sánchez-Sánchez and Izzo, 2018) or large-scale optimization (Kumar et al., 2021). This has motivated the development of models that approximate the input–output behavior of these systems while reducing computational burden (Bhosekar and Ierapetritou, 2018; Iftakher et al., 2022a,b; McBride and Sundmacher, 2019). Surrogate models are increasingly used in chemical process systems, ranging from thermodynamic property predictions to unit operations and full process flowsheet simulation and optimization (Misener and Biegler, 2023).

Among various surrogate modeling strategies, symbolic regression tools like ALAMO (Wilson and Sahinidis, 2017; Cozad et al., 2014) optimize a linear combination of basis functions using mathematical programming, yielding interpretable functional forms. Artificial Neural networks (ANNs) approximate continuous nonlinear functions (Hornik et al., 1989), and have shown exceptional performance across a variety of engineering domains and tasks, including computer vision and language (Krizhevsky et al., 2017; Goodfellow et al., 2020; LeCun et al., 2015; Vaswani et al., 2017), model predictive control (Ren et al., 2022), transport phenomena (Cai et al., 2021a) and structure–property relationships for complex molecules (Iftakher et al., 2025b). However, the black-box nature of ANNs hinders their utility in domains that require physical consistency and interpretability (Kashinath et al., 2021). The unconstrained ANN training often results in predictions that violate known conservation laws, making them unreliable for decision-making in safety-critical applications. To mitigate this, PINNs incorporate physical laws, typically as soft constraints, into the loss function. While soft-constrained PINNs are flexible and generalizable, they suffer from several drawbacks. First, this multi-objective optimization trades prediction accuracy with physical fidelity, but does not guarantee strict satisfaction of the constraints (Raissi et al., 2019, 2020; Cai et al., 2021b). Constraint violations (although penalized)

* Corresponding author at: Artie McFerrin Department of Chemical Engineering, Texas A&M University, College Station, TX 77843-3122, USA.
E-mail address: hasan@tamu.edu (M.M. Faruque Hasan).
[1] Code available at https://github.com/SOULS-TAMU/kkt-hardnet

https://doi.org/10.1016/j.compchemeng.2025.109418
Received 8 July 2025; Received in revised form 20 August 2025; Accepted 22 September 2025
Available online 26 September 2025

can persist, especially when data is scarce or highly nonlinear (Chen et al., 2024; Wang et al., 2022b). This is particularly problematic in engineering applications where surrogate models are cascaded across interconnected unit operations. Even small constraint violations at the component level can accumulate and propagate, undermining the validity of the full system model (Ma et al., 2022). Second, training PINNs is challenging due to the nonconvexity of the objective function landscape induced by soft constraints and the requirement of careful tuning of the penalty parameters (Krishnapriyan et al., 2021; Wang et al., 2022a; Rathore et al., 2024).

To that end, hard-constrained ANNs that embed strict equality and inequality constraints into the model architecture have recently gained increasing attention. Enforcing hard physical constraints can help regularize models in low-data regimes and prevent overfitting (Min et al., 2024; Márquez-Neila et al., 2017). Hard constrained machine learning approaches typically fall into two broad categories: *projection-based* methods and *predict-and-complete* methods. Projection-based methods correct errors due to unconstrained predictions by projecting them onto the feasible region defined by the constraints. This is achieved by solving a distance minimization problem (Chen et al., 2024; Min et al., 2024). In contrast, predict-and-complete architectures generate a subset of the outputs and analytically/numerically complete the rest using constraint equations (Beucler et al., 2021; Donti et al., 2021). Both methods have demonstrated effectiveness in ensuring feasibility during both training and inference. Several notable frameworks have introduced differentiable optimization layers within ANNs to enforce constraints through the Karush-Kuhn-Tucker (KKT) conditions (Nocedal and Wright, 1999). OptNet (Amos and Kolter, 2017) pioneered this idea for quadratic programs, which was later extended to general convex programs with implicit differentiation (Agrawal et al., 2019). These techniques have enabled ANNs to solve constrained optimization problems within the forward pass, allowing gradients to propagate through the solution map. Recent works such as HardNet-Cvx (Min et al., 2024) and DC3 (Donti et al., 2021) have applied several of these ideas to a range of convex and nonlinear problems, leveraging fixed-point solvers and the implicit function theorem to compute gradients. For the special case of linear constraints, closed-form projection layers can be derived analytically, thereby avoiding any iterative schemes to ensure feasibility (Chen et al., 2021, 2024). Recently, Lastrucci and Schweidtmann (Lastrucci and Schweidtmann, 2025) introduced ENFORCE that employs a scalable adaptive differentiable projection module to enforce a set of C^1 nonlinear equality constraints. While most problems of practical importance are nonlinear, limited work exists to ensure hard nonlinear inequality and equality constraints in ANN outputs, especially those involving exponential, polynomial, multilinear, or rational terms in both the inputs and outputs.

In this work, we introduce KKT-Hardnet, a neural network architecture that rigorously enforces hard nonlinear equality and inequality constraints in both input and output variables. KKT-Hardnet ensures constraint satisfaction by solving a square system that arises from the KKT condition of a distance minimization problem (i.e., projection problem). A Newton-type iterative scheme is embedded as a differentiable projection layer. The projection step ensures that the final output satisfies the original nonlinear constraints, with the unconstrained multilayer perceptron (MLP) prediction serving as the initial guess. To handle general nonlinearities without resorting to arbitrary basis expansions, we also demonstrate a symbolic transformation strategy, where nonlinear algebraic expressions are reformulated using a sequence of auxiliary variables, logarithmic transformations, and exponential substitutions. This results in a generalized system of linear and exponential terms. In our formulation, the slack and dual variables for inequality constraints are automatically guaranteed to be nonnegative. We demonstrate the proposed framework on illustrative examples and a pooling problem with nonlinear equality and inequality constraints in both inputs and outputs, as well as using a real-world chemical process simulation problem. Our results show that the proposed approach ensures constraint satisfaction within machine precision or specified tolerance. Embedding hard constraints improves both prediction fidelity and generalization performance when compared to unconstrained MLPs and PINNs with soft constraint regularization.

To summarize, the main contributions of this work are as follows:

- We develop a physics-informed neural network architecture, KKT-Hardnet, for hard constrained machine learning. KKT-Hardnet embeds a projection layer onto a neural network backbone to solve the KKT system of a projection problem via a Newton-type iterative scheme, thereby ensuring hard constraint satisfaction.
- We modify the inequalities to equalities using slack variables and model the complementarity conditions using Fischer–Burmeister reformulation (Jiang, 1997) to ensure nonnegativity of the slack variables and Lagrange multipliers for the inequality constraints.
- We transform general nonlinear equality and inequality constraints into a structure composed of only linear and exponential relations, using log-exponential transformation of nonlinear terms. The transformation allows to isolate all the nonlinearities in a specific linear and exponential form.
- We show that KKT-Hardnet improves surrogate model fidelity compared to vanilla MLPs and soft-constrained PINNs, for example, in applications involving nonlinear thermodynamic equations and process simulations.

The remainder of this paper is organized as follows. Section 2 presents the architecture of KKT-Hardnet, along with the mathematical formulation of the projection mechanism, the KKT system, and the log-exponential transformation. Section 3 presents numerical results for a set of illustrative examples, as well as a highly nonlinear chemical process simulation problem and a pooling problem. Finally, Section 4 provides concluding remarks and directions for future work.

2. Methodology

In many physical systems, the relationship between input variables $\boldsymbol{x}$ and output variables $\boldsymbol{y}$ is governed by known mass and energy conservation laws, inter-variable dependencies, and physical limitations on the operation. These can often be expressed as linear/nonlinear equality or inequality constraints. To rigorously embed such domain knowledge into machine learning, we present KKT-Hardnet, a neural network architecture that guarantees satisfaction of these constraints during both training and inference.

Consider a dataset $\{(\boldsymbol{x}_i, \bar{\boldsymbol{y}}_i)\}_{i=1}^{N}$ representing the input–output behavior of a physical system. The objective is to train a neural network that approximates the underlying functional mapping between the inputs $\boldsymbol{x}$ and the outputs $\bar{\boldsymbol{y}}$. We consider that some prior knowledge about the system is also available in the form of algebraic constraints that relate the inputs $\boldsymbol{x}$ and the outputs $\bar{\boldsymbol{y}}$. Formally, the goal is to learn $\boldsymbol{y} = \mathrm{NN}(\boldsymbol{\Theta}, \boldsymbol{x})$ such that the predicted output $\boldsymbol{y}$ always belongs to the feasible set S, i.e., $\boldsymbol{y} \in S$, where S is defined as $S = \{\boldsymbol{y} | \boldsymbol{h}(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{0}, \boldsymbol{g}(\boldsymbol{x}, \boldsymbol{y}) \leq \boldsymbol{0}\}$. Specifically, $\boldsymbol{h}(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{0}$ represents a set of algebraic equality constraints $h_k(\boldsymbol{x}, \boldsymbol{y}) = 0$ for $k \in \mathcal{N}_E$, and $\boldsymbol{g}(\boldsymbol{x}, \boldsymbol{y}) \leq \boldsymbol{0}$ represents another set of algebraic inequality constraints $g_k(\boldsymbol{x}, \boldsymbol{y}) \leq 0$ for $k \in \mathcal{N}_I$. The functions $\boldsymbol{h}$ and $\boldsymbol{g}$ encode domain-specific knowledge, and may be nonlinear in both $\boldsymbol{x}$ and $\boldsymbol{y}$. Here, all the aggregated tunable parameters (weights and biases) of an ANN model are defined as $\boldsymbol{\Theta}$. We make the following assumptions:

1. The inputs are $\boldsymbol{x} \in \mathbb{R}^m$ and the outputs are $\boldsymbol{y} \in \mathbb{R}^p$.
2. The number of equality constraints $\mathcal{N}_E$ is less than or equal to the number of outputs p of the neural network, i.e., $\mathcal{N}_E \leq p$, to ensure availability of sufficient degrees of freedom for learning from data while satisfying the constraints.
3. The constraint functions $\boldsymbol{h}$ are feasible and linearly independent.

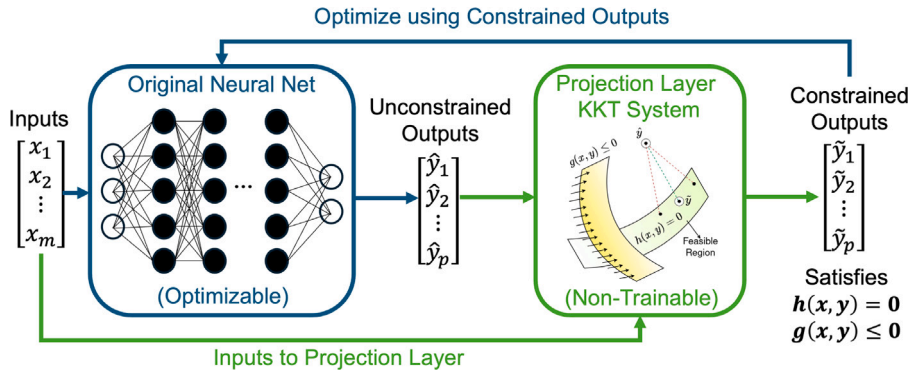

Fig. 1. Neural network architecture of KKT-Hardnet for hard constrained machine learning. Unconstrained outputs are calculated using a standard neural network, while the constrained outputs are calculated as the solution of the system of nonlinear equations corresponding to the KKT condition of a distance minimization problem. The projection layer enforces raw neural network output to lie on the constraint manifold.

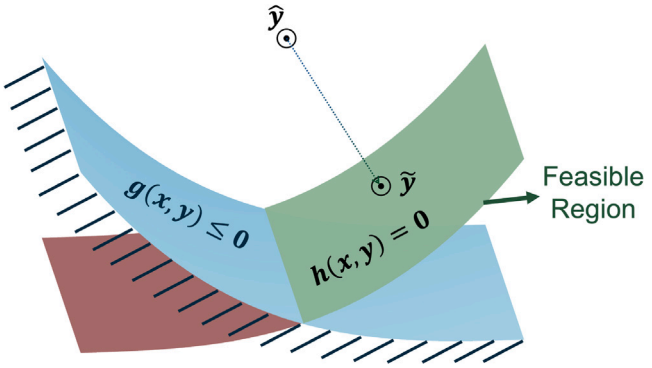

Fig. 2. Projection of neural network outputs onto the constraint manifold.

With these, our approach (see Fig. 1) involves augmenting a standard neural network with a *projection layer* acting as a corrector. Given an input $\boldsymbol{x}$, the network first computes an unconstrained output $\hat{\boldsymbol{y}} = \text{NN}(\boldsymbol{\Theta}, \boldsymbol{x})$, which may violate the physical constraints. The projection layer then adjusts this output to produce a corrected prediction $\tilde{\boldsymbol{y}}$ that lies on the constraint manifold (an illustration is shown in Fig. 2). This strict enforcement of constraints is achieved by solving a square system of nonlinear equations derived from the KKT conditions associated with the following constrained minimization problem:

$$\begin{aligned} \tilde{\boldsymbol{y}}^* = \arg\min_{\boldsymbol{y}} \ & \tfrac{1}{2}\|\boldsymbol{y} - \hat{\boldsymbol{y}}\|^2 \\ \text{s.t.} \ & h_k(\boldsymbol{x}, \boldsymbol{y}) = 0, \quad \forall k \in \mathcal{N}_E, \\ & g_k(\boldsymbol{x}, \boldsymbol{y}) \leq 0, \quad \forall k \in \mathcal{N}_I. \end{aligned} \tag{1}$$

Essentially, this optimization problem searches for a *minimum distance* solution from $\hat{\boldsymbol{y}}$ that lies on the feasible region defined by the equality and inequality constraints. The optimal solution $\tilde{\boldsymbol{y}}^*$ corresponds to a minimum distance projection of $\hat{\boldsymbol{y}}$ onto the feasible region. Due to the presence of nonconvexity, finding $\tilde{\boldsymbol{y}}^*$ requires a global optimization procedure, which may be computationally prohibitive. Therefore, we consider solving the system of nonlinear equations derived from the KKT conditions of (1) that yield a feasible projection $\tilde{\boldsymbol{y}}$. For the special case of convexity, the KKT system is both necessary and sufficient to obtain the minimum distance solution $\tilde{\boldsymbol{y}}^*$. As we describe later, this feasible projection can be directly embedded in the forward pass of a neural network architecture through a Newton-type iterative scheme, ensuring that only physically consistent predictions are used for loss computation and backpropagation. As a result, the loss function for KKT-Hardnet is:

$$\mathcal{L}_{\text{KKT-Hardnet}} = \frac{1}{2N}\sum_{i=1}^{N} \|\tilde{\boldsymbol{y}}_i - \bar{\boldsymbol{y}}_i\|^2, \tag{2}$$

in contrast to the conventional loss function used in standard neural networks:

$$\mathcal{L}_{\text{NN}} = \frac{1}{2N}\sum_{i=1}^{N} \|\hat{\boldsymbol{y}}_i - \bar{\boldsymbol{y}}_i\|^2. \tag{3}$$

Notably, the use of the projected outputs $\tilde{\boldsymbol{y}}$ alter the learning directions during training, which is informed by the feasible manifold. This ultimately guides the model towards parameters that fit the data while respecting the known constraints. This approach is similar to that of Chen et al. (2024) applied to linear equalities. However, we extend it to accommodate both nonlinear equality and inequality constraints. KKT-Hardnet is architecture-agnostic and can be integrated with any neural network backbone, including convolutional neural networks (CNNs) or recurrent neural networks (RNNs).

To enforce the inequality constraints via projection, we introduce non-negative slack variable s_k for $k \in \mathcal{N}_I$ in the optimization problem described by Eq. (1):

$$\begin{aligned} \tilde{\boldsymbol{y}}^* = \arg\min_{\boldsymbol{y}} \ & \tfrac{1}{2}\|\boldsymbol{y} - \hat{\boldsymbol{y}}\|^2 \\ \text{s.t.} \ & h_k(\boldsymbol{x}, \boldsymbol{y}) = 0, \quad \forall k \in \mathcal{N}_E, \\ & g_k(\boldsymbol{x}, \boldsymbol{y}) + s_k = 0, \quad \forall k \in \mathcal{N}_I, \\ & s_k \geq 0, \quad \forall k \in \mathcal{N}_I. \end{aligned} \tag{4}$$

We define the Lagrange multipliers $\mu_k^E \in \mathbb{R}$, $\mu_k^I \geq 0$ for equality and inequality constraints respectively, and write the KKT conditions of the problem in Eq. (4) as follows:

$$\begin{aligned} &\text{(Stationarity)} \qquad \boldsymbol{y} - \hat{\boldsymbol{y}} + \sum_{k\in\mathcal{N}_E} \mu_k^E \nabla_{\boldsymbol{y}} h_k(\boldsymbol{x}, \boldsymbol{y}) \\ &\quad + \sum_{k\in\mathcal{N}_I} \mu_k^I \nabla_{\boldsymbol{y}} g_k(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{0}, \end{aligned} \tag{5}$$

$$\text{(Primal feasibility)} \qquad h_k(\boldsymbol{x}, \boldsymbol{y}) = 0, \quad k \in \mathcal{N}_E \tag{6}$$

$$\text{(Primal feasibility)} \qquad g_k(\boldsymbol{x}, \boldsymbol{y}) + s_k = 0, \quad k \in \mathcal{N}_I \tag{7}$$

$$\text{(Dual feasibility)} \qquad \mu_k^I \geq 0, \quad s_k \geq 0, \quad k \in \mathcal{N}_I \tag{8}$$

$$\text{(Complementarity)} \qquad \mu_k^I \cdot s_k = 0, \quad k \in \mathcal{N}_I. \tag{9}$$

Furthermore, instead of using the original dual feasibility $\mu_k^I \geq 0, s_k \geq 0$ and complementarity condition $\mu_k^I \cdot s_k = 0$, we replace both conditions by the equivalent, Fischer–Burmeister reformulation (Jiang, 1997) as follows.

$$\phi_k(\mu_k^I, s_k): \quad \mu_k^I + s_k - \sqrt{(\mu_k^I)^2 + s_k^2} = 0, \quad k \in \mathcal{N}_I. \tag{10}$$

Note that the Fischer–Burmeister reformulation automatically maintains nonnegativity of the dual and slack variables $\mu_k^I \geq 0,\ s_k \geq 0$ during all iterations. With this reformulation, we are able to generalize the

first-order KKT conditions in a compact form as follows:

$$\begin{aligned} &(F_1)\quad \boldsymbol{y} - \hat{\boldsymbol{y}} + \nabla_{\boldsymbol{y}} \tilde{\boldsymbol{h}}(\boldsymbol{x}, \boldsymbol{y}, \lambda)^{\top} \lambda = 0 \\ &(F_2)\quad \tilde{\boldsymbol{h}}(\boldsymbol{x}, \boldsymbol{y}, \lambda) = \boldsymbol{0} \end{aligned} \tag{11}$$

where, λ is a vector created by concatenation: $\lambda = [\boldsymbol{\mu}^E \quad \boldsymbol{\mu}^I \quad \boldsymbol{s}]$, and $\tilde{\boldsymbol{h}}$ is a vector of functions created by concatenation: $\tilde{\boldsymbol{h}} = [\boldsymbol{h} \quad \boldsymbol{g} + \boldsymbol{s} \quad \boldsymbol{\phi}]$. Therefore, $\tilde{h}_k = 0$ for $k \in \mathcal{N}_T$, where $\mathcal{N}_T$ is the union of constraints given in Eqs. (6), (7) and (10). This completes the first transformation of the KKT conditions as a system of (nonlinear) equalities. Note that Eq. (11) is a square system of equations. Therefore, iterative methods can be directly applied to solve this system of nonlinear equations.

From here, a natural approach is to enforce these KKT conditions by employing an iterative Gauss–Newton procedure. The overall training and inference of KKT-Hardnet are described in Algorithm 1.

Algorithm 1 KKT-Hardnet: Nonlinear Equality and Inequality Constraint Satisfaction via Differentiable Projection

Require: Dataset $\mathcal{D} = \{(\boldsymbol{x}, \bar{\boldsymbol{y}})\}$; neural network backbone $NN_{\boldsymbol{\Theta}} : \mathbb{R}^m \to \mathbb{R}^p$; projection routine $\rho(\cdot)$; Equality/inequality constraints as $\boldsymbol{h}(\boldsymbol{x}, \boldsymbol{y}) = 0$, $\boldsymbol{g}(\boldsymbol{x}, \boldsymbol{y}) \leq 0$; the KKT system corresponding to the projection problem (Eq. (11)).

procedure TRAIN($\mathcal{D}$)
  initialize neural network $NN_{\boldsymbol{\Theta}}$
  while not converged **do**
    for minibatch $\mathcal{B} \subset \mathcal{D}$ **do**
      for $(\boldsymbol{x}, \boldsymbol{y}) \in \mathcal{B}$ **do**
        predict (unconstrained): $\hat{\boldsymbol{y}} \leftarrow NN_{\boldsymbol{\Theta}}(\boldsymbol{x})$
        project to feasible: $\tilde{\boldsymbol{y}} \leftarrow \rho(\hat{\boldsymbol{y}})$
        compute loss: $\mathcal{L}(\tilde{\boldsymbol{y}}, \boldsymbol{y})$
      end for
      update $\boldsymbol{\Theta}$ using $\nabla_{\boldsymbol{\Theta}} \mathcal{L}(\tilde{\boldsymbol{y}}, \boldsymbol{y})$ by backpropagation
    end for
  end while
end procedure

procedure TEST($\boldsymbol{x}, NN_{\boldsymbol{\Theta}}$)
  $\hat{\boldsymbol{y}} \leftarrow NN_{\boldsymbol{\Theta}}(\boldsymbol{x})$
  $\tilde{\boldsymbol{y}} \leftarrow \rho(\hat{\boldsymbol{y}})$
  return $\tilde{\boldsymbol{y}}$
end procedure

Note.
$\rho(\cdot)$ may be (i) a closed-form affine projector when constraints are linear in outputs, or (ii) Gauss–Newton steps on the KKT/log–exp system.

2.1. Implementation of KKT-Hardnet

We define

$$\boldsymbol{F}(\boldsymbol{x}, \hat{\boldsymbol{y}}, \boldsymbol{y}, \lambda) = \begin{pmatrix} F_1(\boldsymbol{x}, \hat{\boldsymbol{y}}, \boldsymbol{y}, \lambda) \\ F_2(\boldsymbol{x}, \boldsymbol{y}, \lambda) \end{pmatrix} = 0,$$

where, F_1 and F_2 are described in Eq. (11). The Jacobian with respect to $(\boldsymbol{y}, \lambda)$ is

$$\boldsymbol{J}(\boldsymbol{y}, \lambda) := \frac{\partial \boldsymbol{F}}{\partial(\boldsymbol{y}, \lambda)}.$$

At iteration k, we solve the Newton step when the Jacobian is invertible as follows:

$$\boldsymbol{J}^{(k)} \begin{bmatrix} \Delta \boldsymbol{y} \\ \Delta \lambda \end{bmatrix} = -\boldsymbol{F}^{(k)},$$

A regularized Gauss–Newton step is also available for the case when the Jacobian is noninvertible:

$$\left[\boldsymbol{J}^{(k)\top} \boldsymbol{J}^{(k)} + \gamma \boldsymbol{I}\right] \begin{bmatrix} \Delta \boldsymbol{y} \\ \Delta \lambda \end{bmatrix} = -\boldsymbol{J}^{(k)\top} \boldsymbol{F}^{(k)},$$

where, $\boldsymbol{J}^{(k)} = \boldsymbol{J}(\boldsymbol{y}^{(k)}, \lambda^{(k)})$ and $\boldsymbol{F}^{(k)} = \boldsymbol{F}(\boldsymbol{x}, \hat{\boldsymbol{y}}, \boldsymbol{y}^{(k)}, \lambda^{(k)})$. Then update

$$\begin{aligned} \boldsymbol{y}^{(k+1)} &= \boldsymbol{y}^{(k)} + \alpha\, \Delta \boldsymbol{y}, \\ \boldsymbol{z}^{(k+1)} &= \boldsymbol{z}^{(k)} + \alpha\, \Delta \boldsymbol{z}, \\ \lambda^{(k+1)} &= \lambda^{(k)} + \alpha\, \Delta \lambda, \end{aligned}$$

where $\alpha \in (0, 1]$ is a step-length parameter (e.g. from Armijo line-search) and $\gamma > 0$ is a small Tikhonov-regularization parameter to ensure invertibility.

During training, we optionally employ an adaptive "warm start" for the Gauss–Newton projection layer. We first train the backbone network alone and monitor the data loss $\mathcal{L}_{\text{NN}}$. When this loss falls below a user–specified threshold η (or after a fixed warm-up budget), we enable the projection layer and continue end-to-end training. Adaptive activation of the projection improves the initial guess $\hat{\boldsymbol{y}}$ supplied to the Gauss–Newton solve, which may reduce the number of projection iterations to converge to the user-specified tolerance.

Differentiation through the projection layer: Starting with $\boldsymbol{y}^{(0)} = \hat{\boldsymbol{y}}$, the projection layer runs a fixed number (K) of Newton/Gauss–Newton steps to enforce the algebraic constraints. During backpropagation, these K iterations are unrolled in the computation graph. Automatic differentiation propagates $\partial \mathcal{L} / \partial \tilde{\boldsymbol{y}}$ backwards through each update $\boldsymbol{y}^{(k)}$ to obtain $\mathrm{d}\mathcal{L}/\mathrm{d}\boldsymbol{\Theta}$ using chain rules. Although unrolling is simple, it is memory-heavy. This is because all intermediate Newton/Gauss–Newton iterates $\{\boldsymbol{y}^{(k)}\}_{k=0}^{K}$ are stored for the backward pass. For a more memory-efficient implementation, one can reduce K, or switch to the implicit/VJP formulation outlined in Appendix B, which needs only a single adjoint solve and does not store all intermediate iterates.

2.2. Log–exponential transformation

A symbolic transformation strategy via logarithmic and exponential substitutions may allow general nonlinear constraints to be rewritten in a structured form where all nonlinearities appear as exponentials and/or linear combinations of variables. Specifically, any constraint of the form $h(\boldsymbol{x}, \boldsymbol{y}) = 0$, that may include nonlinear terms such as y^n, xy, or y/x and so on, can be reformulated into an equivalent system where $\boldsymbol{z}$ denotes a set of new auxiliary variables that allows each nonlinear term to be replaced by an exponential. This yields a constraint system of the general form which is equivalent to the system given by Eq. (11):

$$\boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{y} + \boldsymbol{A}_{\boldsymbol{x}} \exp(\boldsymbol{x}) + \boldsymbol{C}_{\boldsymbol{z}} \boldsymbol{z} + \boldsymbol{C}_{\lambda} \lambda = \boldsymbol{b}, \tag{12a}$$

$$\boldsymbol{D}_{\boldsymbol{y}} \boldsymbol{y} + \boldsymbol{D}_{\boldsymbol{z}} \boldsymbol{z} + \boldsymbol{D}_{\lambda} \lambda = \boldsymbol{d}, \tag{12b}$$

$$\boldsymbol{E}_{\boldsymbol{y}} \boldsymbol{y} + \boldsymbol{E}_{\boldsymbol{z}} \boldsymbol{z} + \boldsymbol{E}_{\lambda} \lambda = \boldsymbol{G} \exp(\boldsymbol{H}_{\boldsymbol{y}} \boldsymbol{y} + \boldsymbol{H}_{\boldsymbol{z}} \boldsymbol{z}). \tag{12c}$$

In general, we can denote this new system of equations in Eq. (12) as $\boldsymbol{F}(\boldsymbol{y}, \boldsymbol{z}, \lambda) = \boldsymbol{0}$. The solution of this system of equations gives $\tilde{\boldsymbol{y}}$. Note that, in this transformed general form, all terms are linear except for $\exp(\boldsymbol{H}_{\boldsymbol{y}} \boldsymbol{y} + \boldsymbol{H}_{\boldsymbol{z}} \boldsymbol{z})$ in Eq. (12c).

To illustrate, consider the nonlinear algebraic constraint:

$$h(\boldsymbol{x}, \boldsymbol{y}) := y_1 - y_2^3 - 12x^2 + 6x - 6 = 0. \tag{13}$$

Let $z_1 := e^{z_5}, z_2 := e^{z_4}, z_3 := y_2^3, \quad z_4 := \log z_3, \quad z_5 := \log y_2$. This implies $z_4 = 3z_5, \quad y_2 = e^{z_5}, \quad z_3 = e^{z_4}$. For inputs, we define: $x_1 := x, \quad x_2 := x^2, \quad x_3 := \log x_2, \quad x_4 := \log x_1$, which implies: $x_3 = 2x_4, \quad x_2 = e^{x_3}, \quad x_1 = e^{x_4}$.

Using these definitions and auxiliary variables, the original constraint (13) can be rewritten as the following system:

$$y_1 - z_3 - 12x_2 + 6x_1 = 6, \tag{14a}$$

$$x_3 - 2x_4 = 0, \tag{14b}$$

$$x_2 - e^{x_3} = 0, \tag{14c}$$

$$x_1 - e^{x_4} = 0. \tag{14d}$$

$$z_4 - 3z_5 = 0, \tag{14e}$$
$$y_2 - z_1 = 0, \tag{14f}$$
$$z_3 - z_2 = 0, \tag{14g}$$
$$z_1 - e^{z_5} = 0, \tag{14h}$$
$$z_2 - e^{z_4} = 0, \tag{14i}$$

where, $\boldsymbol{x} = (x_1, x_2, x_3, x_4)^\top$, $\boldsymbol{y} = (y_1, y_2)^\top$, and $\boldsymbol{z} = (z_1, \dots, z_5)^\top$, with:

$$\boldsymbol{A} = \begin{bmatrix} 6 & -12 & 0 & 0 \\ 0 & 0 & 1 & -2 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\boldsymbol{B} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\boldsymbol{A}_x = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

$$\boldsymbol{C}_z = \begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\boldsymbol{b} = \begin{bmatrix} 6 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\boldsymbol{D}_y = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

$$\boldsymbol{D}_z = \begin{bmatrix} 0 & 0 & 0 & 1 & -3 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \end{bmatrix}$$

$$\boldsymbol{d} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\boldsymbol{E}_z = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\boldsymbol{H}_z = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\boldsymbol{G} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

All other vectors and matrices are zero.

This transformation has several advantages. First, it maps a broad class of algebraic nonlinearities (e.g., products, powers, ratios) into a sparse "linear + exponential" structure that is well suited to Newton-type solvers. Jacobians can easily be computed, and sparsity can be exploited. Second, it enables hard enforcement of nonlinear constraints by solving a structured system with only linear and exponential terms, without the need for an arbitrary number of basis functions. Third, the log–exponential transformation is monotone and preserves nonnegativity for selected variables (e.g., slacks and inequality multipliers), which helps maintain feasibility during line search and step selection, often improving robustness in practice. When combined with the KKT reformulation, the log–exponential transformation naturally enforces nonnegativity of inequality slacks and multipliers, which eliminates the need for active-set type strategies. Also, the "linear + exponential" structure leads to a Jacobian where the only nonlinear blocks are diagonal matrices of exponentials.

Remark 1. log–exponential transformation is applicable for systems with nonnegative outputs. That is, the inputs are $\boldsymbol{x} \in \mathbb{R}^m$ and the outputs are $\boldsymbol{y} \in \mathbb{R}^p_{\geq 0}$.

Remark 2. We interpret all exponentials component-wise. That is, for any vector $\boldsymbol{v}$, $\exp(\boldsymbol{v}) = \left[e^{v_1}, e^{v_2}, \dots\right]^\top$. In particular, $\exp(\boldsymbol{H}_y \boldsymbol{y}) = \begin{bmatrix} e^{(\boldsymbol{H}_y \boldsymbol{y})_1} \\ e^{(\boldsymbol{H}_y \boldsymbol{y})_2} \\ \vdots \end{bmatrix}$, where each entry is the exponential of the corresponding row of $\boldsymbol{H}_y \boldsymbol{y}$.

Remark 3. Lagrange multipliers associated with equality constraints are unrestricted in sign. Therefore, applying a log–exponential transformation to those multipliers forces $\mu^E \geq 0$ and can render the KKT system inconsistent or ill-conditioned in pathological cases.

We consider the following practical options:

(i) Do not log–exp the equality multipliers. Specifically, ensure $\mu^E \in \mathbb{R}$ by applying the log–exp transformation only to the *primal–dual feasibility* conditions and inequality slacks and multipliers, but leave the stationarity equation (where μ^E appears) in its original signed form.

(ii) Rewrite $\boldsymbol{h}(\boldsymbol{x}, \boldsymbol{y}) = 0$ as the pair $\boldsymbol{h}(\boldsymbol{x}, \boldsymbol{y}) \leq 0$ and $-\boldsymbol{h}(\boldsymbol{x}, \boldsymbol{y}) \leq 0$. Then introduce nonnegative slacks, and enforce complementarity with a Fischer–Burmeister reformulation. This yields nonnegative multipliers throughout the KKT system.

(iii) Represent each equality multiplier as a difference of two nonnegative parts, $\mu^E = \mu^+ - \mu^-$ with $\mu^+, \mu^- \geq 0$, and (optionally) apply log–exp to μ^+, μ^-. This preserves unrestrictedness in sign while keeping nonnegativity at the auxiliary variable definitions. However, it increases variables and may be less stable than (i).

We adopt option (i) for numerical experiments involving log–exp transformation unless stated otherwise.

Numerical solution of the log–exponential transformed system of the general KKT conditions: As described above, the log–exponential transformed system corresponding to the KKT conditions for general linear/nonlinear equality and inequality constraints can be represented as Eq. (12). Now define:

$$\mathbf{F}(\boldsymbol{y}, \boldsymbol{z}, \lambda) = \begin{pmatrix} F_1(\boldsymbol{y}, \boldsymbol{z}, \lambda) \\ F_2(\boldsymbol{y}, \boldsymbol{z}, \lambda) \\ F_3(\boldsymbol{y}, \boldsymbol{z}, \lambda) \end{pmatrix} = 0$$

where

$$\begin{aligned} F_1(\boldsymbol{y}, \boldsymbol{z}, \lambda) &:= \boldsymbol{A}\,\hat{\boldsymbol{x}} + \boldsymbol{B}\,\boldsymbol{y} + \boldsymbol{A}_x \exp(\hat{\boldsymbol{x}}) + \boldsymbol{C}_z\,\boldsymbol{z} + \boldsymbol{C}_\lambda\,\lambda - \boldsymbol{b}, \\ F_2(\boldsymbol{y}, \boldsymbol{z}, \lambda) &:= \boldsymbol{D}_y\,\boldsymbol{y} + \boldsymbol{D}_z\,\boldsymbol{z} + \boldsymbol{D}_\lambda\,\lambda - \boldsymbol{d}, \\ F_3(\boldsymbol{y}, \boldsymbol{z}, \lambda) &:= \boldsymbol{E}_y\,\boldsymbol{y} + \boldsymbol{E}_z\,\boldsymbol{z} + \boldsymbol{E}_\lambda\,\lambda - \boldsymbol{G}\exp(\boldsymbol{H}_y\boldsymbol{y} + \boldsymbol{H}_z\boldsymbol{z}). \end{aligned} \tag{15}$$

The Jacobian with respect to $(\boldsymbol{y}, \boldsymbol{z}, \lambda)$ is

$$\boldsymbol{J}(\boldsymbol{y}, \boldsymbol{z}, \lambda) := \frac{\partial \mathbf{F}}{\partial(\boldsymbol{y}, \boldsymbol{z}, \lambda)} = \begin{pmatrix} \boldsymbol{B} & \boldsymbol{C}_z & \boldsymbol{C}_\lambda \\ \boldsymbol{D}_y & \boldsymbol{D}_z & \boldsymbol{D}_\lambda \\ \boldsymbol{E}_y - \boldsymbol{G}\boldsymbol{L}\boldsymbol{H}_y & \boldsymbol{E}_z - \boldsymbol{G}\boldsymbol{L}\boldsymbol{H}_z & \boldsymbol{E}_\lambda \end{pmatrix}, \tag{16}$$

where

$$\boldsymbol{L} := \operatorname{diag}\left(e^{\boldsymbol{H}_y\boldsymbol{y} + \boldsymbol{H}_z\boldsymbol{z}}\right).$$

With this Jacobian, we then apply the Newton/Gauss–Newton procedure (see Section 2.1 for details).

Special Case 1: Equality constraints with nonlinearity in x but linearity in y

If the equality constraints are affine in $\boldsymbol{y}$, then one does not require Newton-type iterative scheme and can derive the projection *analytically* as follows. First, we solve the simplified quadratic program to minimize the Euclidean distance between $\hat{\boldsymbol{y}}$ and $\tilde{\boldsymbol{y}}$ that lies on the constraint manifold.

$$\begin{aligned}\tilde{\boldsymbol{y}} = \ & \arg\min_{\boldsymbol{y}} \ \frac{1}{2}\|\boldsymbol{y}-\hat{\boldsymbol{y}}\|^2, \\ & \text{s.t.} \quad \boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}\boldsymbol{y} + \boldsymbol{A}_{\boldsymbol{x}}\exp(\hat{\boldsymbol{x}}) = \boldsymbol{b},\end{aligned} \tag{17}$$

The Lagrangian can then be constructed as follows:

$$\mathcal{L}(\boldsymbol{y},\lambda) = \frac{1}{2}(\boldsymbol{y}-\hat{\boldsymbol{y}})^T(\boldsymbol{y}-\hat{\boldsymbol{y}}) + \lambda^\top(\boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}\boldsymbol{y} + \boldsymbol{A}_{\boldsymbol{x}}\exp(\hat{\boldsymbol{x}}) - \boldsymbol{b}) \tag{18}$$

Stationary condition:

$$\frac{\partial\mathcal{L}}{\partial\boldsymbol{y}} = (\boldsymbol{y}-\hat{\boldsymbol{y}}) + \boldsymbol{B}^T\lambda = 0 \Rightarrow \boldsymbol{y} = \hat{\boldsymbol{y}} - \boldsymbol{B}^T\lambda \tag{19}$$

Dual feasibility:

$$\begin{aligned}&\frac{\partial\mathcal{L}}{\partial\lambda} = \boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}\boldsymbol{y} + \boldsymbol{A}_{\boldsymbol{x}}\exp(\hat{\boldsymbol{x}}) - \boldsymbol{b} = 0 \\ &\quad \Rightarrow \boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}(\hat{\boldsymbol{y}} - \boldsymbol{B}^T\lambda) + \boldsymbol{A}_{\boldsymbol{x}}\exp(\hat{\boldsymbol{x}}) - \boldsymbol{b} = 0 \\ &\quad \Rightarrow \boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}\hat{\boldsymbol{y}} + \boldsymbol{A}_{\boldsymbol{x}}\exp(\hat{\boldsymbol{x}}) - \boldsymbol{b} = \boldsymbol{B}\boldsymbol{B}^T\lambda\end{aligned} \tag{20}$$

If $\boldsymbol{B}$ is full rank, then $\boldsymbol{B}\boldsymbol{B}^T$ is invertible.

$$\lambda = (\boldsymbol{B}\boldsymbol{B}^T)^{-1}\left(\boldsymbol{B}\hat{\boldsymbol{y}} + \boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{A}_{\boldsymbol{x}}\exp\hat{\boldsymbol{x}} - \boldsymbol{b}\right) \tag{21}$$

$$\begin{aligned}\tilde{\boldsymbol{y}} &= \hat{\boldsymbol{y}} - \boldsymbol{B}^T(\boldsymbol{B}\boldsymbol{B}^T)^{-1}\left(\boldsymbol{B}\hat{\boldsymbol{y}} + \boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{A}_{\boldsymbol{x}}\exp(\hat{\boldsymbol{x}}) - \boldsymbol{b}\right) \\ &= \left[\boldsymbol{I} - \boldsymbol{B}^T(\boldsymbol{B}\boldsymbol{B}^T)^{-1}\boldsymbol{B}\right]\hat{\boldsymbol{y}} \\ &\quad + \left[\boldsymbol{B}^T(\boldsymbol{B}\boldsymbol{B}^T)^{-1}\boldsymbol{A}\right]\hat{\boldsymbol{x}} \\ &\quad + \left[-\boldsymbol{B}^T(\boldsymbol{B}\boldsymbol{B}^T)^{-1}\boldsymbol{A}_{\boldsymbol{x}}\right]\exp(\hat{\boldsymbol{x}}) \\ &\quad + \boldsymbol{B}^T(\boldsymbol{B}\boldsymbol{B}^T)^{-1}\boldsymbol{b} \\ &= \boldsymbol{A}^*\hat{\boldsymbol{x}} + \boldsymbol{B}^*\hat{\boldsymbol{y}} + \boldsymbol{A}^*_{\boldsymbol{x}}\exp(\hat{\boldsymbol{x}}) + \boldsymbol{b}^*\end{aligned} \tag{22}$$

where:

$$\boldsymbol{A}^* = \boldsymbol{B}^\top(\boldsymbol{B}\boldsymbol{B}^\top)^{-1}\boldsymbol{A}, \tag{23a}$$

$$\boldsymbol{B}^* = \boldsymbol{I} - \boldsymbol{B}^\top(\boldsymbol{B}\boldsymbol{B}^\top)^{-1}\boldsymbol{B}, \tag{23b}$$

$$\boldsymbol{A}^*_{\boldsymbol{x}} = -\boldsymbol{B}^\top(\boldsymbol{B}\boldsymbol{B}^\top)^{-1}\boldsymbol{A}_x, \tag{23c}$$

$$\boldsymbol{b}^* = \boldsymbol{B}^\top(\boldsymbol{B}\boldsymbol{B}^\top)^{-1}\boldsymbol{b}. \tag{23d}$$

Special Case 2: Equality constraints with linearity in both x and y

Given a linear constraint $\boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}\boldsymbol{y} = \boldsymbol{b}$, the projection problem becomes:

$$\tilde{\boldsymbol{y}} = \arg\min_{\boldsymbol{y}} \ \frac{1}{2}\|\boldsymbol{y}-\hat{\boldsymbol{y}}\|^2 \quad \text{s.t.} \quad \boldsymbol{A}\hat{\boldsymbol{x}} + \boldsymbol{B}\boldsymbol{y} = \boldsymbol{b} \tag{24}$$

The solution from KKT conditions is:

$$\tilde{\boldsymbol{y}} = \hat{\boldsymbol{y}} - \boldsymbol{B}^\top(\boldsymbol{B}\boldsymbol{B}^\top)^{-1}(\boldsymbol{B}\hat{\boldsymbol{y}} + \boldsymbol{A}\hat{\boldsymbol{x}} - \boldsymbol{b}) \tag{25}$$

This can be written in affine form (Chen et al., 2024):

$$\tilde{\boldsymbol{y}} = \boldsymbol{A}^*\hat{\boldsymbol{x}} + \boldsymbol{B}^*\hat{\boldsymbol{y}} + \boldsymbol{b}^*, \tag{26}$$

where $\boldsymbol{A}^*$, $\boldsymbol{B}^*$, $\boldsymbol{b}^*$ are given by Eqs. (23a), (23b), and (23d) respectively.

To summarize, for constraints involving linear output variables $\boldsymbol{y}$, the KKT conditions allow for an exact, non-iterative projection that can be implemented as a fixed linear layer. However, for general nonlinear constraints defined over both input and output, we resort to Newton-based numerical solution for constraint satisfaction.

3. Numerical experiments

We build all machine learning models using the PyTorch package (Paszke, 2019). We use the Adam optimizer (Kingma, 2014) for training the models. All numerical experiments are performed on a MacBook Pro with an Apple M4 Pro chip (12-core CPU) and 24 GB of RAM, running macOS. The model implementation and case studies are made available on our GitHub repository at: https://github.com/SOULS-TAMU/kkt-hardnet.

Performance metrics: To assess the predictive performance and physical consistency of the models, we report the following metrics: mean squared error (MSE), root mean squared error (RMSE), and mean absolute constraint violation.

For predictions $\hat{\boldsymbol{y}}_i \in \mathbb{R}^p$ and targets $\bar{\boldsymbol{y}}_i \in \mathbb{R}^p$,

$$\text{MSE} = \frac{1}{Np}\sum_{i=1}^{N}\sum_{j=1}^{p}\left(\hat{y}_{ij} - \bar{y}_{ij}\right)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}.$$

Mean absolute constraint violation:

$$\begin{aligned}\text{Violation} = \frac{1}{NT}\sum_{i=1}^{N}\Bigg(&\sum_{k\in\mathcal{N}_E}\left|h_k(\boldsymbol{x}_i,\hat{\boldsymbol{y}}_i)\right| \\ &+ \sum_{k\in\mathcal{N}_I}\text{ReLU}\left(g_k(\boldsymbol{x}_i,\hat{\boldsymbol{y}}_i)\right)\Bigg), \qquad T := |\mathcal{N}_E| + |\mathcal{N}_I|,\end{aligned} \tag{27}$$

where $\text{ReLU}(t) = \max\{t,0\}$. Thus, inequality terms contribute 0 when $g_k(\boldsymbol{x}_i,\hat{\boldsymbol{y}}_i) \le 0$ and contribute their magnitude when violated.

3.1. Example 1: Nonlinearity in both input and output

We consider the scalar input feature $x \in [1,2]$ and a nonlinear vector-valued target function

$$\boldsymbol{y}(x) = \begin{bmatrix} y_1(x) \\ y_2(x)\end{bmatrix} = \begin{bmatrix} 8x^3+5 \\ 2x-1\end{bmatrix}, \qquad \boldsymbol{y}: \ \mathbb{R} \longrightarrow \mathbb{R}^2. \tag{28}$$

with $n_{\text{train}} = 1200$ and $n_{\text{val}} = 300$. In addition to data $\{(x_i, \boldsymbol{y}(x_i))\}_{i=1}^{n}$, we assume knowledge of the following algebraic constraint:

$$h(x,\boldsymbol{y}) := y_1 - y_2^3 - 12x^2 + 6x - 6 = 0. \tag{29}$$

The Lagrangian of the projection problem becomes:

$$\mathcal{L}(y_1,y_2,\lambda) = \tfrac{1}{2}(y_1-\hat{y}_1)^2 + \tfrac{1}{2}(y_2-\hat{y}_2)^2 + \lambda\, h(x,\boldsymbol{y}) \tag{30a}$$

The KKT system becomes:

$$\frac{\partial\mathcal{L}}{\partial y_1} \ : \ y_1 - \hat{y}_1 + \lambda = 0, \tag{31a}$$

$$\frac{\partial\mathcal{L}}{\partial y_2} \ : \ y_2 - \hat{y}_2 - 3\,\lambda\, y_2^2 = 0, \tag{31b}$$

$$\frac{\partial\mathcal{L}}{\partial\lambda} \ : \ y_1 - y_2^3 - 12\,x^2 + 6\,x - 6 = 0. \tag{31c}$$

Following the definitions and log–exponential transformations in Section 2.1, the KKT system becomes:

$$y_1 - \hat{y}_1 + \lambda = 0, \tag{32a}$$

$$y_2 - \hat{y}_2 - 3\lambda y_2^2 = 0, \tag{32b}$$

$$y_1 - z_3 - 12x_2 + 6x_1 = 6, \tag{32c}$$

$$x_3 - 2x_4 = 0, \tag{32d}$$

$$x_2 - e^{x_3} = 0, \tag{32e}$$

$$x_1 - e^{x_4} = 0. \tag{32f}$$

$$z_4 - 3z_5 = 0, \tag{32g}$$

$$y_2 - z_1 = 0, \tag{32h}$$

$$z_3 - z_2 = 0, \tag{32i}$$

$$z_1 - e^{z_5} = 0, \tag{32j}$$

Table 1
Regression accuracy and constraint violation of KKT-Hardnet compared with an MLP and soft-constrained PINN for example 1.

Model	Training		Validation	
	MSE	$\|h\|$	MSE	$\|h\|$
MLP	1.361×10^{2}	9.61	1.235×10^{2}	9.13
PINN	6.445×10^{2}	1.27×10^{-1}	6.066×10^{2}	1.24×10^{-1}
KKT-Hardnet	8.253×10^{1}	4.21×10^{-8}	7.585×10^{1}	3.50×10^{-8}

$$z_2 - e^{z_4} = 0, \tag{32k}$$

where, $\hat{y}_1$ and $\hat{y}_2$ are unconstrained neural network predictions.

Next, we apply Algorithm 1 to seek the closest point on the manifold constructed by equalities in (32). We specify thirty Newton steps ($K = 30$) with tolerance $\|\boldsymbol{F}\|_\infty < 10^{-6}$. The projector $\rho(\hat{\boldsymbol{y}}) = (\tilde{y}_1^{(K)}, \tilde{y}_2^{(K)})$ is fully differentiable, enabling end-to-end training. We compare three different strategies for handling algebraic constraints: (i) an unconstrained multilayer perception (MLP), (ii) a soft constrained PINN, and (iii) KKT-Hardnet. Vanilla MLP involves two hidden layers $[1 \to 64 \to 64 \to 2]$ with ReLU activation function. KKT-Hardnet consists of the same backbone as the MLP, followed by the projection layer. For PINN, with an identical backbone to MLP, the loss is augmented by a soft penalty $\mathcal{L} = \text{MSE} + \omega(y_1 - y_3 - 12x^2 + 6x - 6)^2$. All models are trained with Adam ($\text{lr} = 10^{-4}$) for 1200 epochs. The PINN uses penalty weight $\omega = 100$.

Fig. 3 shows the learning curves for normalized RMSE and absolute constraint violation. The KKT-Hardnet converges nearly an order of magnitude faster and enforces the constraint down to numerical precision, whereas the PINN only reduces the violation to $\mathcal{O}(10^{-1})$ despite the large penalty weight. Table 1 summarizes performance on both training and validation data. KKT-Hardnet reduces constraint violation by 7–8 orders of magnitude while maintaining lower MSE than the unconstrained baseline. In contrast, PINN incurs large MSE errors due to the difficulty of soft constraint enforcement.

3.2. Example 2: Nonlinearity in input only

We consider a two-dimensional input $\boldsymbol{x} = (x_1, x_2) \in [1, 2]^2$, and define a vector-valued ground truth function $\boldsymbol{y}(\boldsymbol{x}) = (y_1(\boldsymbol{x}), y_2(\boldsymbol{x}))^\top$ as:

$$\boldsymbol{y}(x_1, x_2) = \begin{bmatrix} x_1^2 + x_2^2 \\ 4x_1^2 + 4x_2^3 - 2x_2^2 \end{bmatrix}, \qquad \boldsymbol{y} : \mathbb{R}^2 \to \mathbb{R}^2. \tag{33}$$

We assume the following constraint holds:

$$y_1 + \tfrac{1}{2} y_2 = 3x_1^2 + 2x_2^3. \tag{34}$$

We then introduce auxiliary variables:

$$x_3 = x_1^2, \qquad x_4 = \log x_1, \qquad x_5 = \log x_3, \tag{35a}$$
$$x_6 = x_2^3, \qquad x_7 = \log x_2, \qquad x_8 = \log x_6. \tag{35b}$$

This gives the following log-transformed system:

$$y_1 + \tfrac{1}{2} y_2 - 3x_3 - 2x_6 = 0, \tag{36a}$$
$$2x_4 - x_5 = 0, \tag{36b}$$
$$3x_7 - x_8 = 0, \tag{36c}$$
$$x_1 - e^{x_4} = 0, \tag{36d}$$
$$x_3 - e^{x_5} = 0, \tag{36e}$$
$$x_2 - e^{x_7} = 0, \tag{36f}$$
$$x_6 - e^{x_8} = 0. \tag{36g}$$

We collect all eight inputs as $\boldsymbol{x} = (x_1, \ldots, x_8)^\top \in \mathbb{R}^8$, and express the constraint system in the general form (see Eq. (12a)) as follows:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 0 & -3 & 0 & 0 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & -1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}, \quad \boldsymbol{B} = \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix},$$

$$\boldsymbol{A}_{\boldsymbol{x}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{bmatrix}.$$

Analytic Projection: We notice that, only the first row of $\boldsymbol{B}$ depends on the output, corresponding to:

$$\boldsymbol{B}_{\text{active}} \boldsymbol{y} = 3x_3 + 2x_6, \quad \text{with} \quad \boldsymbol{B}_{\text{active}} = \begin{bmatrix} 1 & \frac{1}{2} \end{bmatrix}.$$

Rows 2 to 7 do not depend on $\boldsymbol{y}$. Therefore, for any given input x_1, x_2, all terms in rows 2 to 7 are already satisfied, and thus, they do not participate in the projection. The projection of an unconstrained neural network prediction $\hat{\boldsymbol{y}}$ onto this hyperplane is therefore, given by the analytic form:

$$\tilde{\boldsymbol{y}} = \hat{\boldsymbol{y}} - \boldsymbol{B}_{\text{active}}^\top \left(\boldsymbol{B}_{\text{active}} \boldsymbol{B}_{\text{active}}^\top \right)^{-1} \left(\boldsymbol{B}_{\text{active}} \hat{\boldsymbol{y}} - 3x_3 - 2x_6 \right). \tag{37}$$

Since $\boldsymbol{B}_{\text{active}} \boldsymbol{B}_{\text{active}}^\top = \frac{5}{4}$, the scalar residual r can be defined as:

$$r = \left(\boldsymbol{B}_{\text{active}} \hat{\boldsymbol{y}} - 3x_3 - 2x_6 \right)$$
$$= \hat{y}_1 + 0.5\, \hat{y}_2 - 3x_3 - 2x_6, \qquad \text{and} \qquad \left(\boldsymbol{B}_{\text{active}} \boldsymbol{B}_{\text{active}}^\top \right)^{-1} = \tfrac{4}{5}.$$

The final projection update becomes:

$$\tilde{\boldsymbol{y}} = \hat{\boldsymbol{y}} - \frac{4}{5} \begin{bmatrix} 1 \\ \frac{1}{2} \end{bmatrix} r. \tag{38}$$

$$\tilde{y}_1 = \hat{y}_1 - 0.8\, r, \qquad \tilde{y}_2 = \hat{y}_2 - 0.4\, r. \tag{39}$$

Note that, if one wishes to consider the entire $\boldsymbol{B} \in \mathbb{R}^{7\times 2}$, the analytic projection does not change, except $\boldsymbol{B}\boldsymbol{B}^\top$ becomes singular. In that case, one needs to replace the regular inverse with a Moore–Penrose pseudo-inverse (Barata and Hussein, 2012). The general projection formula collapses exactly to the same update as given in Eq. (39), because all the zero rows in $\boldsymbol{B}$ drop out of the pseudoinverse calculation.

Training results: We train three models using identical MLP architectures $[2 \to 64 \to 64 \to 2]$ with ReLU activations and the Adam optimizer ($\text{lr} = 10^{-4}$, 1200 epochs). Model 1 is MLP which is an unconstrained neural network trained via MSE loss, Model 2 is PINN where MSE loss with soft penalty $\omega(y_1 + 0.5y_2 - (3x_1^2 + 2x_2^3))$, $\omega = 100$ is applied. Finally, Model 3 is KKT-Hardnet where analytic projection, i.e., Eq. (39), is applied after raw prediction from the MLP backbone. As shown in Fig. 4 and Table 2, KKT-Hardnet reduces constraint violation by over 6 orders of magnitude while achieving the lowest validation error. In contrast, the soft-penalty PINN model suffers from large errors due to inadequate constraint enforcement.

3.3. Example 3: Inequality constraints

We demonstrate the hard enforcement of inequality with the task of learning $y = x^2$ subject to the scalar inequality $y - x \leq 0$. We generate data by uniformly sampling x such that,

$$x_i \sim \mathcal{U}(1, 2), \qquad y_i = x_i^2, \qquad \begin{cases} i = 1, \ldots, 1200 & \text{(training data)} \\ i = 1201, \ldots, 1500 & \text{(validation data)} \end{cases} \tag{40}$$

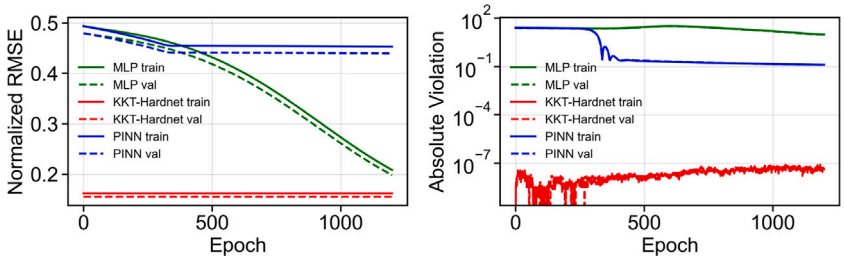

Fig. 3. Learning curves for example 1. Left: Normalized RMSE (RMSE is normalized by the range of outputs), Right: constraint violation over 1200 epochs.

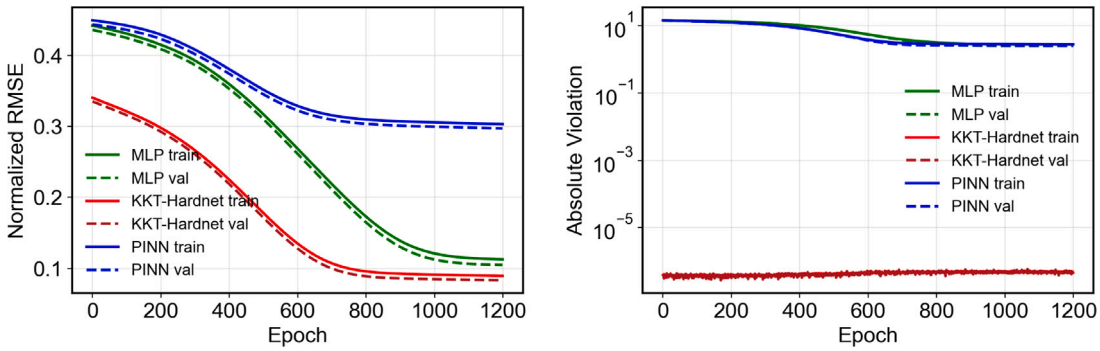

Fig. 4. Learning curves for example 2. Left: Normalized RMSE; Right: absolute constraint violation over 1200 epochs.

Table 2
Regression accuracy and constraint enforcement comparison for the illustrative example 2.

Model	Training		Validation	
	MSE	$\|h\|$	MSE	$\|h\|$
MLP	1.443×10^{1}	2.77	1.248×10^{1}	2.51
PINN	1.045×10^{2}	2.81	1.004×10^{2}	2.55
KKT-Hardnet	9.051×10^{0}	4.60×10^{-7}	7.863×10^{0}	4.23×10^{-7}

We fit a vanilla MLP $\hat{y} = \mathrm{NN}_{\boldsymbol{\Theta}}(x) \in \mathbb{R}$ and then project $\hat{y}$ onto $\{y \mid y \le x\}$ by performing the following projection:

$$\min_{y} \quad \tfrac{1}{2}(y - \hat{y})^2 \tag{41}$$

$$\text{s.t.} \quad y - x + s = 0, \tag{42}$$

$$s \ge 0 \tag{43}$$

The KKT system is then derived as follows:

$$y - \hat{y} + \mu^E = 0, \tag{44}$$

$$\mu^E - \mu^I = 0, \tag{45}$$

$$y - x + s = 0, \tag{46}$$

$$\mu^I s = 0, \tag{47}$$

$$\mu^I \ge 0, \tag{48}$$

$$s \ge 0 \tag{49}$$

Next, we use FischerBurmeister reformulation and define the following auxiliary variables for the log–exponential transformation:

$$z_3 = \log(\mu^I), \qquad z_5 = \log(s), \tag{50a}$$

$$z_1 = {\mu^I}^2, \qquad z_2 = s^2, \tag{50b}$$

$$z_7 = z_1 + z_2, \qquad z_4 = 2\, z_3, \tag{50c}$$

$$z_6 = 2\, z_5, \qquad z_{10} = \log(z_7), \tag{50d}$$

$$z_9 = \tfrac{1}{2}\, z_{10}, \qquad z_8 = e^{z_9}. \tag{50e}$$

These auxiliary variables during log–exponential transformation ensure that at initialization, all logarithmic and exponential relations hold exactly, and that $\mu^I > 0$, $s > 0$ throughout the Newton iterations. At the start of each forward pass we set

$$y = \hat{y}, \qquad \mu^E = 0, \tag{51a}$$

$$\mu^I = \mathrm{ReLU}(\hat{y} - x) + \epsilon, \tag{51b}$$

$$s = \mathrm{ReLU}(x - \hat{y}) + \epsilon. \tag{51c}$$

with $\epsilon = 10^{-3}$.

After the reformulation, the KKT system becomes

$$F_1(\boldsymbol{\tau}) : \; y - \hat{y} + \mu^E = 0, \tag{52a}$$

$$F_2(\boldsymbol{\tau}) : \; \mu^E - \mu^I = 0, \tag{52b}$$

$$F_3(\boldsymbol{\tau}) : \; y - x + s = 0, \tag{52c}$$

$$F_4(\boldsymbol{\tau}) : \; z_8 - \mu^I - s = 0, \tag{52d}$$

$$F_5(\boldsymbol{\tau}) : \; \mu^I - e^{z_3} = 0, \tag{52e}$$

$$F_6(\boldsymbol{\tau}) : \; z_1 - e^{z_4} = 0, \tag{52f}$$

$$F_7(\boldsymbol{\tau}) : \; 2\, z_3 - z_4 = 0, \tag{52g}$$

$$F_8(\boldsymbol{\tau}) : \; s - e^{z_5} = 0, \tag{52h}$$

$$F_9(\boldsymbol{\tau}) : \; z_2 - e^{z_6} = 0, \tag{52i}$$

$$F_{10}(\boldsymbol{\tau}) : \; 2\, z_5 - z_6 = 0, \tag{52j}$$

$$F_{11}(\boldsymbol{\tau}) : \; z_7 - z_1 - z_2 = 0, \tag{52k}$$

$$F_{12}(\boldsymbol{\tau}) : \; z_9 - \tfrac{1}{2}\, z_{10} = 0, \tag{52l}$$

$$F_{13}(\boldsymbol{\tau}) : \; z_8 - e^{z_9} = 0, \tag{52m}$$

$$F_{14}(\boldsymbol{\tau}) : \; z_7 - e^{z_{10}} = 0. \tag{52n}$$

where all the variables are compactly represented by a vector $\boldsymbol{\tau} = [\boldsymbol{y}, \boldsymbol{z}, \lambda]$ as follows:

$$\boldsymbol{\tau} = \left[y,\, s,\, \mu^I,\, \mu^E,\, z_3,\, z_5,\, z_1,\, z_2,\, z_7,\, z_4,\, z_6,\, z_8,\, z_9,\, z_{10}\right]^\top$$

These residuals are solved by a damped Gauss–Newton iteration to specified tolerance, yielding the projected output $\tilde{y}$ (see Algorithm 1).

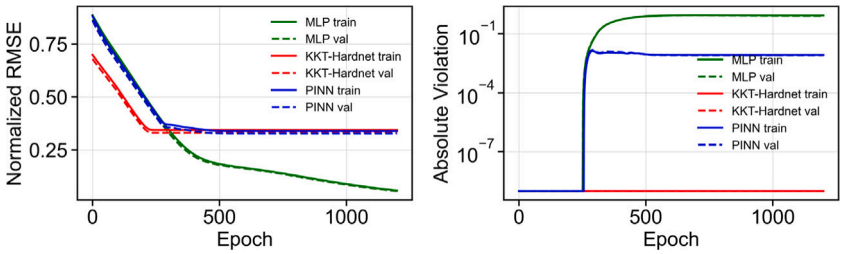

Fig. 5. Learning curves for example 3. Left: Normalized RMSE; Right: absolute constraint violation (log scale).

Table 3
Regression accuracy and constraint enforcement comparison for the illustrative example 3.

Model	Training		Validation	
	MSE	$\|g\|$	MSE	$\|g\|$
MLP	3.038×10^{-2}	8.48×10^{-1}	2.767×10^{-2}	8.22×10^{-1}
PINN	1.037×10^{0}	8.32×10^{-3}	9.648×10^{-1}	8.00×10^{-3}
KKT-Hardnet	1.059×10^{0}	1.00×10^{-9}	9.857×10^{-1}	1.00×10^{-9}

Model training and comparison. We compare KKT-Hardnet with an unconstrained MLP and a soft-constrained PINN, all using identical architectures and training routines. Fig. 5 shows the normalized RMSE and absolute constraint violation for all three models. The results in Table 3 clearly show that KKT-Hardnet maintains constraint satisfaction up to numerical precision while achieving comparable MSE to the unconstrained MLP. In contrast, the PINN suffers from persistent constraint violations despite soft penalty enforcement ($\omega = 100$).

3.4. A case study on chemical process simulation involving nonlinear thermodynamics

We consider the simulation of an extractive distillation process for separating the azeotropic refrigerant mixture R-410A using the ionic liquid [EMIM][SCN]. The flowsheet is given in Fig. 6, which consists of a distillation column with an ionic liquid entrainer fed at a fixed column stage. The feed R-410A is an azeotropic mixture of R-32 and R-125. Therefore, the mass composition of R-32 and R-125 in R-410A feed are 0.5 and 0.5, respectively. [EMIM][SCN] and R-410A are assumed to be fed at stage 2 and 11 respectively. The total number of stages of the column is set to 18. Due to the presence of ionic liquid as entrainer, highly nonlinear thermodynamics is present for the calculation of phase equilibria across the extractive distillation column. In this case study, all simulations are performed using equilibrium-stage modeling using NRTL model in Aspen Plus® v12. Details on modeling and simulation of the extractive distillation process can be found elsewhere (Iftakher et al., 2025b,a; Monjur et al., 2022).

3.4.1. Problem setup and data generation

We vary three input variables: total feed molar flow rate (F_{F}), ionic liquid flow rate (F_{IL}), and the reflux ratio (RR), and extract a set of steady-state outputs from Aspen Plus. The input space is sampled on a uniform grid: (1) Feed flow rate: $F_{\mathrm{F}} \in [95, 110]$ kg/h, (2) IL flow rate: $F_{\mathrm{IL}} \in [800, 950]$ kg/h, and (3) Reflux ratio: $RR \in [2.5, 4.3]$. The corresponding outputs include: (1) Distillate and bottoms flowrates: $F_{\mathrm{D}}, F_{\mathrm{B}}$ [mol/s], (2) Tray liquid flowrate: L [mol/s], (3) Mole fractions in distillate and bottoms: $x^{D}_{R32}, x^{D}_{R125}, x^{D}_{IL}, x^{B}_{R32}, x^{B}_{R125}, x^{B}_{IL}$. Not all simulations converged without error. Therefore, we postprocessed all simulations, and filtered out unconverged ones. This resulted in 4308 simulation data points, which constitute our dataset.

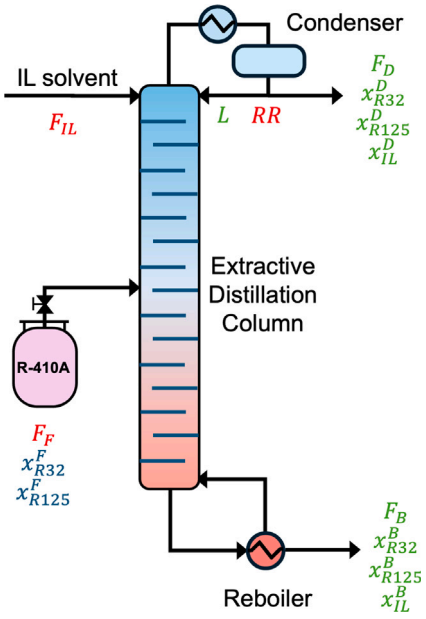

Fig. 6. Extractive distillation of R-410A using an Ionic liquid [EMIM][SCN]. The input variables are denoted in red. The output variables are denoted in green. The fixed parameters are denoted in blue. The column operates at a fixed pressure of 10 bar.

3.4.2. Nonlinear constraints in both input and output

We consider six physical constraints, derived from mass and energy balances involving mixed linearity and nonlinearities in both inputs and outputs:

(C1) Overall molar balance (affine):

$$F_{\mathrm{F}} + F_{\mathrm{IL}} = F_{\mathrm{D}} + F_{\mathrm{B}} \tag{53}$$

(C2) Component balance — R32 (nonlinear in input–output):

$$F_{\mathrm{F}}\, x^{\mathrm{F}}_{\mathrm{R32}} = F_{\mathrm{D}}\, x^{\mathrm{D}}_{\mathrm{R32}} + F_{\mathrm{B}}\, x^{\mathrm{B}}_{\mathrm{R32}} \tag{54}$$

(C3) Component balance — R125 (nonlinear in input–output):

$$F_{\mathrm{F}}\, x^{\mathrm{F}}_{\mathrm{R125}} = F_{\mathrm{D}}\, x^{\mathrm{D}}_{\mathrm{R125}} + F_{\mathrm{B}}\, x^{\mathrm{B}}_{\mathrm{R125}} \tag{55}$$

(C4) Mole fraction balance — Distillate (affine):

$$x^{\mathrm{D}}_{\mathrm{R32}} + x^{\mathrm{D}}_{\mathrm{R125}} + x^{\mathrm{D}}_{\mathrm{IL}} = 1 \tag{56}$$

(C5) Mole fraction closure — Bottoms (affine):

$$x^{\mathrm{B}}_{\mathrm{R32}} + x^{\mathrm{B}}_{\mathrm{R125}} + x^{\mathrm{B}}_{\mathrm{IL}} = 1 \tag{57}$$

(C6) Reflux ratio relation (bilinear in input and output):

$$RR = \frac{L}{F_{\mathrm{D}}} \tag{58}$$

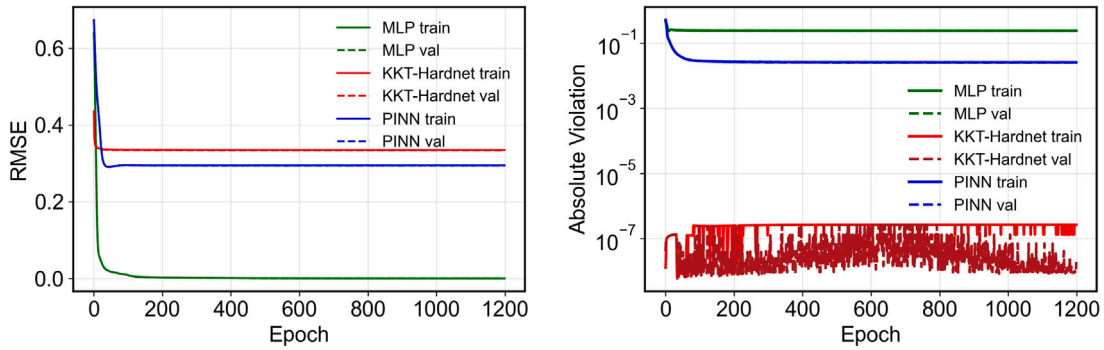

Fig. 7. Learning curves for the simulation of extractive distillation-based simulation of R-410A using an ionic liquid [EMIM][SCN]. Left: RMSE; Right: absolute constraint violation over 1200 epochs.

Table 4
Model performance comparison on the extractive distillation case study.

Model	Training		Validation	
	MSE	$\|h\|$	MSE	$\|h\|$
MLP	3.146×10^{-8}	2.39×10^{-1}	3.798×10^{-8}	2.39×10^{-1}
PINN	8.680×10^{-2}	2.55×10^{-2}	8.662×10^{-2}	2.52×10^{-2}
KKT-Hardnet	1.119×10^{-1}	2.70×10^{-7}	1.113×10^{-1}	1.95×10^{-8}

Here, $x^F_{R32} = 0.6976$ and $x^F_{R125} = 0.3024$.

Next, we enforce the six nonlinear algebraic constraints by constructing the KKT system and log–exponential transformation (see Section 2.2 for details). Specifically, bilinear terms like $F_D \cdot x^D_{R32}$ are log transformed and replaced by auxiliary variables, allowing the constraint system to be rewritten in the form given in Eq. (12). Then we train KKT-Hardnet (see Algorithm 1). In our implementation, we use $K = 30$, $\gamma = 10^{-3}$; and $\alpha \in (0, 1]$ is adjusted based on backtracking and Armijo condition. The projection operation is fully differentiable and acts as a corrector layer: $\rho(\hat{\boldsymbol{y}}) = \tilde{\boldsymbol{y}}^{(K)}$.

We compare the performance of KKT-Hardnet with a soft penalty-based PINN and an unconstrained MLP, all using identical architectures and optimization settings (learning rate 10^{-4}, 1200 epochs, Adam optimizer). The PINN model includes the six physical constraints as penalty terms in the loss function with a penalty weight $\omega = 10.0$. Fig. 7 shows the learning curves for RMSE and absolute constraint violation. We observe no violation spikes above 10^{-6}, confirming that both training and validation outputs satisfy the constraints to the specified tolerance. The small oscillations below 10^{-6} arise because, in some forward passes, the Gauss–Newton iteration terminates with the residual norm dropping well below the stopping criterion of 10^{-6}. In those cases, the final residual may lie anywhere between the tolerance (10^{-6}) and the numerical precision limit. Thus, the fluctuations reflect solver termination under finite precision.

As shown in Table 4, the KKT-Hardnet achieves constraint satisfaction up to the specified tolerance ($\sim 10^{-6}$). Although its MSE is higher than the unconstrained MLP, it eliminates physical inconsistency and constraint violation. In contrast, the PINN, despite a soft penalty, fails to sufficiently reduce constraint violation, indicating that soft regularization alone is insufficient for strict feasibility in this nonlinear setting. An analysis of the PINN penalty weight ω on MSE and constraint violation is given in Appendix A.

These results support the following observation: MLP tries to minimize the loss between the prediction and the raw data (even if the raw data may not satisfy known physics). On the other hand, KKT-Hardnet tries to minimize the difference between a physics-satisfying output and the raw data. Therefore, it may be that the RMSE for KKT-Hardnet is larger than MLP. However, the constraint violation is eliminated. PINN, on the other hand, offers a tradeoff (higher loss than MLP but lower violation than MLP).

3.4.3. Analytic projection for affine-in-output constraints

We consider a special case which consists of a subset of four constraints that are linear in outputs. These include the overall molar balance, the mole fraction balances for distillate and bottoms, and the reflux relation. Specifically:

(C1) Overall molar balance:

$$F_F + F_{IL} = F_D + F_B \tag{59}$$

(C4) Mole fraction closure — Distillate:

$$x^D_{R32} + x^D_{R125} + x^D_{IL} = 1 \tag{60}$$

(C5) Mole fraction closure — Bottoms:

$$x^B_{R32} + x^B_{R125} + x^B_{IL} = 1 \tag{61}$$

(C6) Reflux ratio relation:

$$L = RR \cdot F_D \tag{62}$$

These constraints define a linear manifold over the 9-dimensional output space. Given a raw neural network prediction $\hat{\boldsymbol{y}} \in \mathbb{R}^9$, we project it onto the feasible manifold using an analytic projection given in Eq. (22):

$$\boldsymbol{y}_{\text{proj}} = \hat{\boldsymbol{y}} - \boldsymbol{B}^\top (\boldsymbol{B}\boldsymbol{B}^\top)^{-1} \boldsymbol{r}, \tag{63}$$

where $\boldsymbol{B} \in \mathbb{R}^{4\times 9}$ encodes the output coefficients of the constraints, and $\boldsymbol{r} \in \mathbb{R}^4$ is the residual vector formed from the violation of each constraint. Specifically, for a given input $\boldsymbol{x} = (F_F, F_{IL}, RR)$, the residual vector is computed as:

$$\boldsymbol{r} = \begin{bmatrix} F_F + F_{IL} - (\hat{F}_D + \hat{F}_B) \\ \hat{x}^D_{R32} + \hat{x}^D_{R125} + \hat{x}^D_{IL} - 1 \\ \hat{x}^B_{R32} + \hat{x}^B_{R125} + \hat{x}^B_{IL} - 1 \\ -R \cdot \hat{F}_D + \hat{L} \end{bmatrix} \tag{64}$$

This projection is performed independently for each sample in the batch, and requires no iterative solver or Jacobian computation. The matrix $\boldsymbol{B}$ is fixed for each sample based on known input $\boldsymbol{x}$. As such, the projection step is differentiable and computationally efficient.

We compare KKT-Hardnet with an analytic projection layer against an unconstrained MLP and a soft-constrained PINN, all using identical architectures and training routines. Fig. 8 shows the RMSE and constraint violation for all three models. The results in Table 5 clearly show that the analytically projected KKT-Hardnet maintains constraint satisfaction up to numerical precision while achieving lower MSE than the unconstrained MLP. In contrast, the PINN suffers from persistent

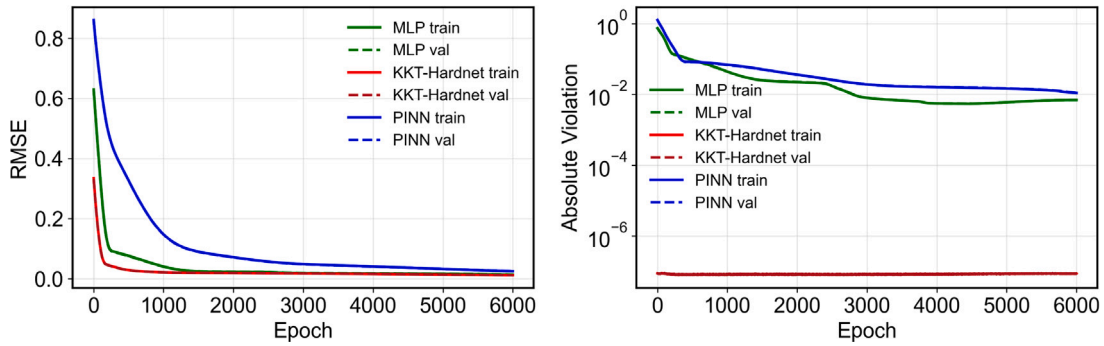

Fig. 8. Learning curves for the analytic projection case. Left: RMSE; Right: absolute constraint violation (log scale).

Table 5
Model performance for affine-in-output constraints for the extractive distillation case study.

Model	Training		Validation	
	MSE	$\|h\|$	MSE	$\|h\|$
MLP	1.828×10^{-4}	6.89×10^{-3}	1.932×10^{-4}	6.90×10^{-3}
PINN	6.359×10^{-4}	1.09×10^{-2}	6.505×10^{-4}	1.11×10^{-2}
KKT-Hardnet	1.390×10^{-4}	8.66×10^{-8}	1.479×10^{-4}	8.61×10^{-8}

constraint violations despite soft penalty enforcement. An analysis of the PINN penalty weight ω on MSE and constraint violation is given in Appendix A.

This case highlights that when constraints are affine or linear in outputs, an exact and efficient analytic projection can be used to enforce strict feasibility without iterative solvers. In our dataset, the Aspen Plus simulations already satisfy these four constraints to near machine precision (maximum residuals $\approx 10^{-9}$). In such a case, KKT-Hardnet guides the backpropagation and alters the learning direction such that it offers much faster convergence than MLP. It also eliminates constraint violation.

3.5. A case study on pooling problem involving nonlinear equality and inequality constraints

We consider a pooling problem discussed in Floudas et al. (2013) where both nonlinear equality and inequality constraints are present. The flowsheet is given in Fig. 9, which consists of a pool, one splitter and two mixture. The product streams X and Y are produced by combining the feed steams A, B and C with sulfur content of 3%, 1% and 2% respectively. The maximum allowable sulfur contents in product X and Y are 2.5% and 1.5%, respectively. These specifications yield the inequality constraints. The nonlinearity arises because A and B must be pooled together. In this case study, we define m to be the percentage of sulfur content of the streams coming out of the pool.

3.5.1. Problem setup and data generation

We vary four input variables: (1) feed flowrate A, (2) feed flowrate B, (3) product flowrate X, (4) product flowrate Y. The input space is sampled on a random grid: $A, B \in [0, 500]$, $X \in [0, 100]$ and $Y \in [0, 200]$. The corresponding outputs include: (1) Percent amount of sulfur in the streams coming out of the pool (m), (2) flowrates of the streams coming from pool (P_x, P_y) and (3) splitted flowrates of Feed C to the mixers (C_x, C_y). We generate 2000 data points for training the model.

3.5.2. Nonlinear constraints: Equality and inequality

We consider four equality constraints derived from the material balances and two inequality constraints derived from the product specification requirements regarding the sulfur content.

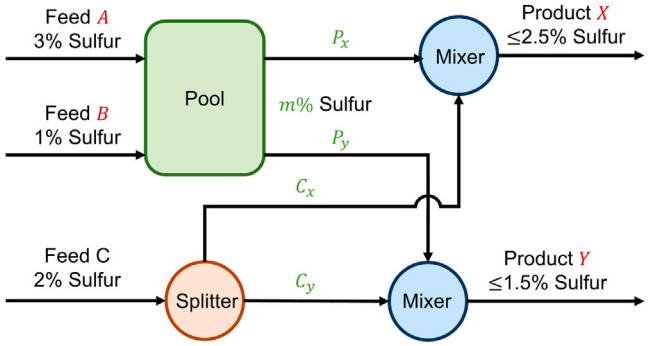

Fig. 9. Process flowsheet of a pooling problem. The product can be formed from three feeds. The input variables are denoted in red. The output variables are denoted in green. Feed compositions are fixed. Feed A and B must be pooled together.

(C1) Pool mass balance (affine equality):

$$P_x + P_y = A + B \tag{65}$$

(C2) Mixer mass balance for X (affine equality):

$$X = P_x + C_x \tag{66}$$

(C3) Mixer mass balance for Y(affine equality):

$$Y = P_y + C_y \tag{67}$$

(C4) Sulfur balance in pool (nonlinear equality):

$$mP_x + mP_y = 3A + B \tag{68}$$

(C5) Product Specification X (nonlinear inequality):

$$mP_x + 2C_x \leq 2.5X \tag{69}$$

(C6) Product Specification Y (nonlinear inequality):

$$mP_y + 2C_y \leq 1.5X \tag{70}$$

In this case study, we enforce the six algebraic constraints, both equality and inequality, by constructing the KKT system (without log–exponential transformation). In our implementation, we use $K = 100, \gamma = 10^{-2}$, and fixed $\alpha = 0.5$ (no backtracking).

We compare the performance of KKT-Hardnet with PINN and MLP, all using identical architectures and optimization settings (learning rate 10^{-3}, 1200 epochs, Adam optimizer). The PINN model includes the constraints as a penalty term in the loss function with a penalty weight $\omega = 1.0$. The learning curves for RMSE and absolute constraint violation are provided in Fig. 10. The RMSE plot shows a spike around the 25^{th} epoch for KKT-Hardnet, which is due to the activation of the

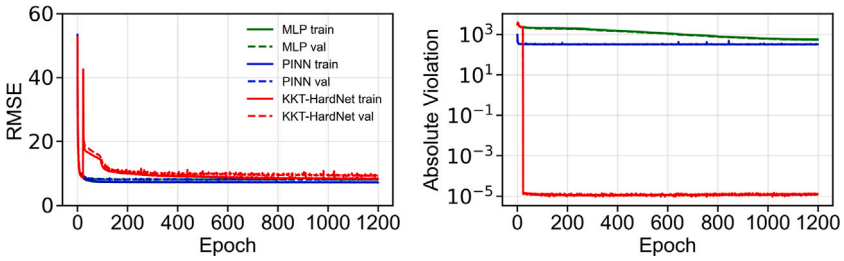

Fig. 10. Learning curves for the pooling problem case study. Left: RMSE; Right: absolute constraint violation over 1200 epochs.

Table 6
Model performance comparison on the pooling problem case study.

Model	Training		Validation	
	MSE	$\|h\| + \|g\|$	MSE	$\|h\| + \|g\|$
MLP	53.537	5.58×10^{2}	67.758	5.51×10^{2}
PINN	56.69	3.00×10^{1}	71.32	2.94×10^{1}
KKT-Hardnet	74.92	1.08×10^{-5}	92.257	1.05×10^{-5}

projection layer (in this case, we demonstrate the use of the adaptive activation of the projection layer), and eventually, the error drops close to MLP and PINN error. We observe constraint violations around 10^{-5}. Note that the data was not standardized, which leads to a higher numerical precision error. Even in this case, compared to MLP and PINN, KKT-Hardnet reduces the constraint violation by 6–7 orders of magnitude.

Table 6 provides the MSE and absolute constraint violation for both training and testing data. Constraint satisfaction around 10^{-5} is achieved for KKT-Hardnet. Though the MSE for KKT-Hardnet is higher than MLP and PINN, it is able to significantly reduce the constraint violation.

4. Conclusions

We have presented a physics-informed neural network architecture that enforces hard nonlinear equality and inequality constraints in both inputs and outputs through a differentiable projection layer. The projection involves solving a square system of nonlinear equations corresponding to the KKT conditions of a distance minimization problem. The KKT system of equations is then subjected to a Newton-type iterative scheme, which acts as a differentiable projection layer within the neural network and guarantees constraint satisfaction. In doing so, we have addressed the limitations of traditional PINNs that rely on soft constraint penalty parameters and lack guaranteed satisfaction of first-principles-based governing equations. Furthermore, we have introduced a log–exponential transformation of a wide class of nonlinear equations (involving polynomial, power, rational, etc.) to a structured form involving linear and exponential terms. Numerical experiments on illustrative examples, a pooling problem, and a highly nonlinear chemical process simulation problem demonstrate that the proposed approach eliminates constraint violations to within specified tolerance and machine precision. It also improves predictive accuracy compared to conventional MLPs and soft-constrained PINNs. Future directions include extending the method for application to differential–algebraic systems, learning the solutions of nonlinear optimization problems, and devising tailored algorithm that exploits the structure and sparsity of the Jacobian for the log–exponential system.

CRediT authorship contribution statement

Ashfaq Iftakher: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization, Software. **Rahul Golder:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization, Data curation, Software. **Bimol Nath Roy:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Writing – original draft, Writing – review & editing. **M.M. Faruque Hasan:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge partial funding support from the NSF, USA CAREER award CBET-1943479 and the EPA Project Grant 84097201. Part of the research was conducted with the computing resources provided by Texas A&M High Performance Research Computing.

Appendix A. Effect of the penalty weight on PINN training

We study the influence of the soft-constraint penalty weight w in PINN for the extractive distillation case study described in Section 3.4. We rerun the experiments and report a shaded band across $w \in [0.1, 100]$ sweep showing the epoch-wise min–max envelope with the mean curve for both RMSE and mean absolute violation $|h|$. Note that the exact MSE and violation values may slightly differ across different random seeds because the training does not follow the same optimization path due to the nature of the stochastic gradient descent. Also, the DataLoader shuffling in different runs can yield a different minibatch order, which may prompt different weights in each epoch, resulting in a slightly different Newton-projection convergence. However, as described next, the conclusions are consistent.

Nonlinear constraints in both input and output. This system corresponds to the problem definition and constraints described in Section 3.4.2. The unconstrained MLP achieves low data error but violates known constraints substantially (val MSE $\approx 4.3 \times 10^{-8}$ versus $|h| \approx 2.39 \times 10^{-1}$). KKT-Hardnet attains near-zero violation ($|h| \approx 1.9 \times 10^{-7}$) with moderate data error (val MSE $\approx 1.11 \times 10^{-1}$). The PINN (as shown in Fig. 11) exhibits the expected Pareto trade-off. As w increases, violation decreases monotonically while MSE increases. With $w = 0.1$ we obtain the best validation MSE (2.40×10^{-3}) but poor constraint satisfaction ($|h| = 2.05 \times 10^{-1}$). On the other hand, with $w = 100$ the violation drops to $|h| = 3.42 \times 10^{-3}$ at the cost of an increased validation MSE 1.08×10^{-1}. Detailed results are given in Table 7.

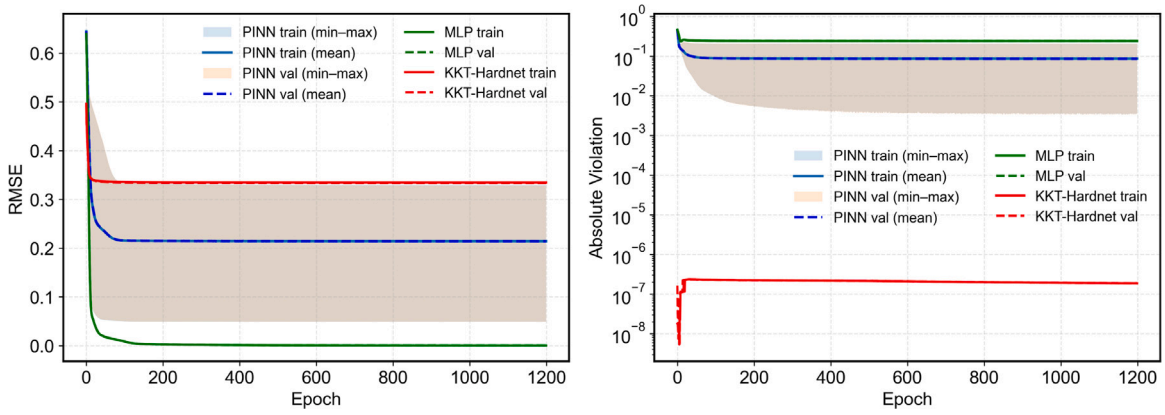

Fig. 11. Training curves comparing MLP, KKT-Hardnet, and PINNs with a sweep over $w \in [0.1, 100]$. Left: RMSE; right: mean absolute constraint violation. Shaded regions show the min–max envelope across w with a solid (train) and dashed (val) mean.

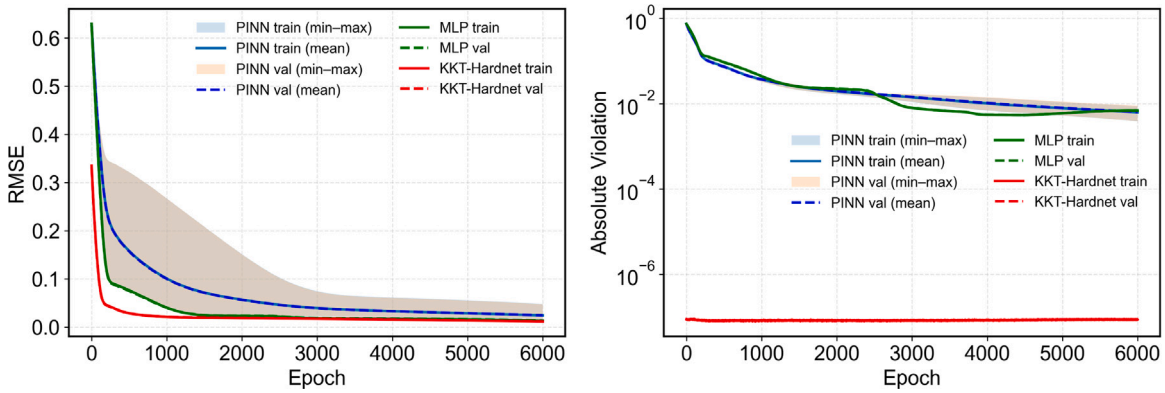

Fig. 12. Training curves comparing MLP, KKT-Hardnet, and PINNs with a sweep over $w \in [0.1, 100]$. Left: RMSE; right: mean absolute constraint violation. Shaded regions show the min–max envelope across w with a solid (train) and dashed (val) mean.

Table 7
PINN performance across different w for the extractive distillation case study with nonlinear input–output constraints.

Model	Training		Validation	
	MSE	$\|h\|$	MSE	$\|h\|$
MLP	3.390×10^{-8}	2.39×10^{-1}	4.302×10^{-8}	2.39×10^{-1}
KKT-Hardnet	1.120×10^{-1}	1.87×10^{-7}	1.112×10^{-1}	1.87×10^{-7}
PINN $\lambda = 0.1$	2.368×10^{-3}	2.05×10^{-1}	2.397×10^{-3}	2.05×10^{-1}
PINN $\lambda = 1$	3.355×10^{-2}	1.10×10^{-1}	3.377×10^{-2}	1.09×10^{-1}
PINN $\lambda = 10$	8.674×10^{-2}	2.57×10^{-2}	8.656×10^{-2}	2.54×10^{-2}
PINN $\lambda = 100$	1.085×10^{-1}	3.42×10^{-3}	1.079×10^{-1}	3.42×10^{-3}

Affine-in-output constraints. This system corresponds to the problem definition and constraints described in Section 3.4.3. Here, the effect of w is not pronounced, and the spread across w is small in the final epochs (see the narrow violation band in Fig. 12). KKT-Hardnet again achieves negligible violation ($\sim 8.6 \times 10^{-8}$) and, notably, better data fit than the MLP (validation MSE 1.48×10^{-4} vs. 1.93×10^{-4}). For the PINN, the best validation MSE occurs at $w = 0.1$ while the lowest violation is reached near $w = 1$ (see Table 8). This case thus highlights the practical difficulty of choosing a single penalty weight for PINN that simultaneously optimizes data fit and constraint feasibility. Soft penalties in PINN require tuning w to navigate a Pareto front that is problem-dependent.

Table 8
PINN performance across different w for the extractive distillation case study with affine-in-output constraints.

Model	Training		Validation	
	MSE	$\|h\|$	MSE	$\|h\|$
MLP	1.828×10^{-4}	6.89×10^{-3}	1.932×10^{-4}	6.90×10^{-3}
KKT-Hardnet	1.390×10^{-4}	8.66×10^{-8}	1.479×10^{-4}	8.61×10^{-8}
PINN $\lambda = 0.1$	2.899×10^{-4}	8.79×10^{-3}	3.061×10^{-4}	8.96×10^{-3}
PINN $\lambda = 0.3$	2.906×10^{-4}	6.72×10^{-3}	3.066×10^{-4}	6.84×10^{-3}
PINN $\lambda = 1$	3.056×10^{-4}	$\mathbf{3.85 \times 10^{-3}}$	3.221×10^{-4}	$\mathbf{3.88 \times 10^{-3}}$
PINN $\lambda = 3$	4.163×10^{-4}	5.92×10^{-3}	4.347×10^{-4}	6.02×10^{-3}
PINN $\lambda = 10$	4.886×10^{-4}	5.80×10^{-3}	5.062×10^{-4}	5.89×10^{-3}
PINN $\lambda = 30$	8.743×10^{-4}	6.17×10^{-3}	8.756×10^{-4}	6.22×10^{-3}
PINN $\lambda = 100$	2.340×10^{-3}	6.28×10^{-3}	2.271×10^{-3}	6.36×10^{-3}

Appendix B. Implicit differentiation for KKT-Hardnet

Instead of backpropagating through all Newton steps, one can differentiate the solution of the KKT system directly. To illustrate this, recall that the KKT residuals are collected as

$$\boldsymbol{F}(\boldsymbol{y}, \lambda; \boldsymbol{x}, \hat{\boldsymbol{y}}) = \begin{bmatrix} \boldsymbol{y} - \hat{\boldsymbol{y}} + \nabla_{\boldsymbol{y}} \tilde{\boldsymbol{h}}(\boldsymbol{x}, \boldsymbol{y}, \lambda)^{\top} \lambda \\ \tilde{\boldsymbol{h}}(\boldsymbol{x}, \boldsymbol{y}, \lambda) \end{bmatrix} = \boldsymbol{0}, \tag{71}$$

where $\tilde{\boldsymbol{h}}$ contains both equalities and the inequality reformulation. Define

$$\boldsymbol{J}_s := \frac{\partial \boldsymbol{F}}{\partial(\boldsymbol{y},\boldsymbol{\lambda})}, \qquad \boldsymbol{J}_{\hat{y}} := \frac{\partial \boldsymbol{F}}{\partial \hat{\boldsymbol{y}}}, \qquad \boldsymbol{J}_x := \frac{\partial \boldsymbol{F}}{\partial \boldsymbol{x}}.$$

Differentiating $\boldsymbol{F} = 0$ with respect to any variable $t \in \{\boldsymbol{\Theta}, \boldsymbol{x}\}$ gives

$$\boldsymbol{J}_s \begin{bmatrix} \frac{\partial \tilde{\boldsymbol{y}}}{\partial t} \\ \frac{\partial \boldsymbol{\lambda}}{\partial t} \end{bmatrix} + \boldsymbol{J}_{\hat{y}} \frac{\partial \hat{\boldsymbol{y}}}{\partial t} + \boldsymbol{J}_x \frac{\partial \boldsymbol{x}}{\partial t} = \boldsymbol{0}. \tag{72}$$

By the chain rule, $\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}t} = \left(\frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{y}}}\right)^\top \frac{\partial \tilde{\boldsymbol{y}}}{\partial t}$.

We introduce the adjoint $\boldsymbol{v} = [\boldsymbol{v}_y; \boldsymbol{v}_\lambda]$ as the solution of the transpose–KKT system

$$\boldsymbol{J}_s^\top \boldsymbol{v} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{y}}} \\ \boldsymbol{0} \end{bmatrix}. \tag{73}$$

Left-multiplying (72) by $\boldsymbol{v}^\top$ yields the vector–Jacobian product (VJP) form

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}t} = -\boldsymbol{v}^\top \left(\boldsymbol{J}_{\hat{y}} \frac{\partial \hat{\boldsymbol{y}}}{\partial t} + \boldsymbol{J}_x \frac{\partial \boldsymbol{x}}{\partial t} \right). \tag{74}$$

In our formulation, the stationarity block (71) contains $+\boldsymbol{y} - \hat{\boldsymbol{y}}$, hence

$$\boldsymbol{J}_{\hat{y}} = \frac{\partial}{\partial \hat{\boldsymbol{y}}} \begin{bmatrix} \boldsymbol{y} - \hat{\boldsymbol{y}} \\ \boldsymbol{0} \end{bmatrix} = \begin{bmatrix} -\boldsymbol{I} \\ \boldsymbol{0} \end{bmatrix}.$$

Therefore,

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{\Theta}} = \boldsymbol{v}_y^\top \frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{\Theta}} \qquad \text{and} \qquad \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{x}} = -\boldsymbol{v}^\top \boldsymbol{J}_x + \boldsymbol{v}_y^\top \frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{x}}. \tag{75}$$

Numerical and implementation notes. *Unrolled Newton/Gauss–Newton* lets autograd backpropagate through every iteration and linear solve. This is simple but memory-heavy, because all intermediate states of the K iterations must be kept for the backward pass. The *Implicit adjoint* method discussed above is memory-light. This is because, at the backward pass, only the single transpose–KKT system is solved, which is then backpropagated through the MLP backbone. If the forward projection uses a ridge $\gamma > 0$ in the normal equations, using the same γ in the adjoint (e.g., solving $(\boldsymbol{J}_s^\top \boldsymbol{J}_s + \gamma \boldsymbol{I})\boldsymbol{v}_y = \partial \mathcal{L}/\partial \tilde{\boldsymbol{y}}$ after eliminating $\boldsymbol{v}_\lambda$) may improve robustness for nearly singular constraints.

Data availability

Data will be made available on request.

References

Agrawal, A., Amos, B., Barratt, S., Boyd, S., Diamond, S., Kolter, J.Z., 2019. Differentiable convex optimization layers. Adv. Neural Inf. Process. Syst. 32.

Amos, B., Kolter, J.Z., 2017. Optnet: Differentiable optimization as a layer in neural networks. In: International Conference on Machine Learning. PMLR, pp. 136–145.

Barata, J.C.A., Hussein, M.S., 2012. The Moore–Penrose pseudoinverse: A tutorial review of the theory. Braz. J. Phys. 42, 146–165.

Beucler, T., Pritchard, M., Rasp, S., Ott, J., Baldi, P., Gentine, P., 2021. Enforcing analytic constraints in neural networks emulating physical systems. Phys. Rev. Lett. 126 (9), 098302.

Bhosekar, A., Ierapetritou, M., 2018. Advances in surrogate based modeling, feasibility analysis, and optimization: A review. Comput. Chem. Eng. 108, 250–267, URL: https://www.sciencedirect.com/science/article/pii/S0098135417303228.

Cai, S., Mao, Z., Wang, Z., Yin, M., Karniadakis, G.E., 2021a. Physics-informed neural networks (PINNs) for fluid mechanics: A review. Acta Mech. Sin. 37 (12), 1727–1738.

Cai, S., Wang, Z., Wang, S., Perdikaris, P., Karniadakis, G.E., 2021b. Physics-informed neural networks for heat transfer problems. J. Heat Transf. 143 (6), 060801.

Chen, H., Flores, G.E.C., Li, C., 2024. Physics-informed neural networks with hard linear equality constraints. Comput. Chem. Eng. 189, 108764.

Chen, Y., Huang, D., Zhang, D., Zeng, J., Wang, N., Zhang, H., Yan, J., 2021. Theory-guided hard constraint projection (HCP): A knowledge-based data-driven scientific machine learning method. J. Comput. Phys. 445, 110624.

Cozad, A., Sahinidis, N.V., Miller, D.C., 2014. Learning surrogate models for simulation-based optimization. AIChE J. 60 (6), 2211–2227.

Donti, P.L., Rolnick, D., Kolter, J.Z., 2021. DC3: A learning method for optimization with hard constraints. arXiv preprint arXiv:2104.12225.

Eivazi, H., Tahani, M., Schlatter, P., Vinuesa, R., 2022. Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations. Phys. Fluids 34 (7).

Floudas, C.A., Pardalos, P.M., Adjiman, C., Esposito, W.R., Gümüs, Z.H., Harding, S.T., Klepeis, J.L., Meyer, C.A., Schweiger, C.A., 2013. Handbook of Test Problems in Local and Global Optimization, vol. 33, Springer Science & Business Media.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y., 2020. Generative adversarial networks. Commun. ACM 63 (11), 139–144.

Hornik, K., Stinchcombe, M., White, H., 1989. Multilayer feedforward networks are universal approximators. Neural Netw. 2 (5), 359–366.

Iftakher, A., Aras, C.M., Monjur, M.S., Hasan, M.M.F., 2022a. Data-driven approximation of thermodynamic phase equilibria. AIChE J. 68 (6), e17624.

Iftakher, A., Aras, C.M., Monjur, M.S., Hasan, M.M.F., 2022b. Guaranteed error-bounded surrogate modeling and application to thermodynamics. In: Computer Aided Chemical Engineering, vol. 49, Elsevier, pp. 1831–1836.

Iftakher, A., Leonard, T., Hasan, M.M.F., 2025a. Integrating different fidelity models for process optimization: A case of equilibrium and rate-based extractive distillation using ionic liquids. Comput. Chem. Eng. 192, 108890.

Iftakher, A., Monjur, M.S., Leonard, T., Gani, R., Hasan, M.F., 2025b. Multiscale high-throughput screening of ionic liquid solvents for mixed-refrigerant separation. Comput. Chem. Eng. 199, 109138.

Jiang, H., 1997. Smoothed Fischer-Burmeister equation methods for the complementarity problem. Dep. Math. Univ. Melb. (Aust.).

Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Yang, L., 2021. Physics-informed machine learning. Nat. Rev. Phys. 3 (6), 422–440.

Kashinath, K., Mustafa, M., Albert, A., Wu, J., Jiang, C., Esmaeilzadeh, S., Azizzadenesheli, K., Wang, R., Chattopadhyay, A., Singh, A., et al., 2021. Physics-informed machine learning: case studies for weather and climate modelling. Phil. Trans. R. Soc. A 379 (2194), 20200093.

Kingma, D.P., 2014. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.

Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., Mahoney, M.W., 2021. Characterizing possible failure modes in physics-informed neural networks. Adv. Neural Inf. Process. Syst. 34, 26548–26560.

Krizhevsky, A., Sutskever, I., Hinton, G.E., 2017. ImageNet classification with deep convolutional neural networks. Commun. ACM 60 (6), 84–90.

Kumar, P., Rawlings, J.B., Wright, S.J., 2021. Industrial, large-scale model predictive control with structured neural networks. Comput. Chem. Eng. 150, 107291.

Lastrucci, G., Schweidtmann, A.M., 2025. ENFORCE: Exact nonlinear constrained learning with adaptive-depth neural projection. arXiv preprint arXiv:2502.06774.

LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. Nature 521 (7553), 436–444.

Ma, K., Sahinidis, N.V., Amaran, S., Bindlish, R., Bury, S.J., Griffith, D., Rajagopalan, S., 2022. Data-driven strategies for optimization of integrated chemical plants. Comput. Chem. Eng. 166, 107961.

Márquez-Neila, P., Salzmann, M., Fua, P., 2017. Imposing hard constraints on deep networks: Promises and limitations. arXiv preprint arXiv:1706.02025.

McBride, K., Sundmacher, K., 2019. Overview of surrogate modeling in chemical process engineering. Chem. Ing. Tech. 91 (3), 228–239, URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/cite.201800091.

Min, Y., Sonar, A., Azizan, N., 2024. Hard-constrained neural networks with universal approximation guarantees. arXiv preprint arXiv:2410.10807.

Misener, R., Biegler, L., 2023. Formulating data-driven surrogate models for process optimization. Comput. Chem. Eng. 179, 108411.

Monjur, M.S., Iftakher, A., Hasan, M.M.F., 2022. Separation process synthesis for high-gwp refrigerant mixtures: Extractive distillation using ionic liquids. Ind. Eng. Chem. Res. 61 (12), 4390–4406.

Nocedal, J., Wright, S.J., 1999. Numerical Optimization. Springer.

Paszke, A., 2019. Pytorch: An imperative style, high-performance deep learning library. arXiv preprint arXiv:1912.01703.

Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J. Comput. Phys. 378, 686–707.

Raissi, M., Yazdani, A., Karniadakis, G.E., 2020. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. Science 367 (6481), 1026–1030.

Rathore, P., Lei, W., Frangella, Z., Lu, L., Udell, M., 2024. Challenges in training PINNs: A loss landscape perspective. arXiv preprint arXiv:2402.01868.

Ren, Y.M., Alhajeri, M.S., Luo, J., Chen, S., Abdullah, F., Wu, Z., Christofides, P.D., 2022. A tutorial review of neural network modeling approaches for model predictive control. Comput. Chem. Eng. 165, 107956.

Sánchez-Sánchez, C., Izzo, D., 2018. Real-time optimal control via deep neural networks: study on landing problems. J. Guid. Control Dyn. 41 (5), 1122–1135.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Wang, C., Li, S., He, D., Wang, L., 2022a. Is L^2 physics informed loss always suitable for training physics informed neural network? Adv. Neural Inf. Process. Syst. 35, 8278–8290.

Wang, S., Yu, X., Perdikaris, P., 2022b. When and why PINNs fail to train: A neural tangent kernel perspective. J. Comput. Phys. 449, 110768.

Wilson, Z.T., Sahinidis, N.V., 2017. The ALAMO approach to machine learning. Comput. Chem. Eng. 106, 785–795.